uc3m | Universidad Carlos III de Madrid

Master Degree in Cybersecurity

Academic Year: 2024-2025

# *Master Thesis* "The Game Beneath the Game: The Evolution of Cheats and Anti-Cheat Systems in Counter-Strike 2"

---

Pablo Valiente

Tutor: Antonio Nappa

Universidad Carlos III de Madrid

Date: June 2025

**Abstract**

This thesis examines the evolution of cheating in video games, from early debugging tools to modern external and internal exploits. It explores key cheating techniques, anti-cheat countermeasures, and the ongoing arms race between both sides. Additionally, a practical implementation of a cheat in **Counter-Strike 2** is developed to assess the effectiveness of contemporary anti-cheat systems and identify potential vulnerabilities.

**Keywords:** Cheating, Video Games, Counter-Strike 2, Anti-Cheat Systems, Game Hacking, Memory Manipulation

# DEDICATION

# CONTENTS

# LIST OF FIGURES

# LIST OF TABLES

# 1. INTRODUCTION

## 1 Context and Motivation

The evolution of video games from a niche entertainment medium to a dominant digital industry has led to the rise of **competitive gaming**, where players engage in skill-based challenges to outperform their opponents. The widespread availability of high-speed internet and global connectivity has further transformed online multiplayer games into structured **competitive ecosystems**, characterized by rankings, tournaments, and financial incentives that continuously drive players to refine their strategies and skills.

Today, video games are an integral part of millions of lives worldwide, serving not only as entertainment but also as social platforms, economic engines, and professional career paths. However, the increasing competitiveness and monetization of gaming have also fueled the prevalence of **cheating**, where players exploit unauthorized modifications to gain an unfair advantage. These methods range from memory manipulation and external assistance tools to network-based exploits and hardware modifications. The ongoing **arms race** between game developers—who continuously strengthen anti-cheat mechanisms—and cheat developers—who evolve their techniques to bypass these defenses—has made cheating one of the most pressing challenges in modern gaming.

At the same time, the implementation of anti-cheat systems introduces significant **privacy, legal, and environmental dilemmas**. Advanced detection methods often require deep system access, raising concerns over user privacy and digital rights. Meanwhile, the large-scale commercialization of cheats results in substantial financial losses for game developers and complex regulatory challenges. Additionally, the computational cost of sophisticated anti-cheat solutions contributes to increased energy consumption, raising further environmental concerns.

To go beyond theoretical discussions and ethical concerns, this research will delve into the **technical intricacies of cheating** by implementing and analyzing a custom-built cheat within a large-scale multiplayer game. This approach will allow for an in-depth evaluation of modern anti-cheat systems, providing empirical insights into their strengths, weaknesses, and potential bypasses.

By combining historical analysis, technical research, and practical implementation, this study aims to offer a comprehensive understanding of cheating in competitive gaming. A case study in a large-scale multiplayer game will serve as the foundation for assessing the effectiveness of contemporary anti-cheat measures and their broader societal impact.

## 2 Objectives and Scope

This study aims to analyze cheating in competitive video games from both a theoretical and practical perspective. It will explore its historical evolution, technical mechanisms, and countermeasures, while also assessing the effectiveness of modern anti-cheat systems through hands-on experimentation.

To achieve this, the research will focus on three key areas:

1. **The evolution of cheating in competitive gaming**, examining its impact on game design, player behavior, and industry responses.
2. **A technical analysis of prevalent cheating techniques**, including aimbots, wallhacks, network manipulation, and memory injection, alongside their implications for competitive integrity.
3. **The implementation and evaluation of a custom cheat in a large-scale multiplayer game**, providing empirical insights into the strengths and weaknesses of current anti-cheat solutions.

## 3 Competencies and Alignment with the Master's Program

This thesis aligns with the core competencies of the **Master's Degree in Cybersecurity**, equipping professionals with essential skills in **reverse engineering, intrusion analysis, and digital forensic investigation**. The key competencies developed in this study are:

**Basic Competencies**

- **[CB6]** Conducting original research on competitive gaming security, analyzing cheating techniques and countermeasures.
- **[CB7]** Applying cybersecurity principles to game security, focusing on **cheat development, detection, and evasion**.
- **[CB8]** Evaluating the ethical and legal considerations of game manipulation, with implications for software security and fair competition.

**General Competencies**

- **[CG1]** Investigating and replicating game manipulation techniques to assess vulnerabilities in competitive gaming environments.
- **[CG2]** Examining the role of anti-cheat systems in detecting unauthorized modifications and analyzing their effectiveness.

- **[CG3]** Producing well-documented research on offensive security methodologies, including memory hacking and system intrusion.

**Specific Competencies**

- **[CE1]** Analyzing the psychological motivations behind cheating in competitive gaming.
- **[CE2]** Investigating modern cheat evasion techniques and their ability to bypass detection.
- **[CE3]** Researching trends in game security threats and drawing parallels to broader cybersecurity challenges.
- **[CE4]** Conducting forensic analysis of game environments to extract digital evidence of unauthorized modifications.
- **[CE6]** Assessing the architecture and performance of modern anti-cheat systems, evaluating their strengths and weaknesses.

## 4 Document Structure

This thesis is structured as follows:

- **Chapter 1: Introduction** – Provides the research context, outlining the motivation for the study, its main objectives and scope, the competencies addressed, and the alignment with the Master's program. It also introduces the structure of the document.
- **Chapter 2: From Primitive Survival to Digital Domination** – Explores the human drive for competition from an evolutionary and psychological perspective. It traces the historical evolution of computation, the birth of the internet, and the emergence of online environments as new competitive arenas—culminating in the rise of online gaming as a virtual battlefield.
- **Chapter 3: The Evolution of Cheating: From Debugging Tools to an Organized Industry** – Chronicles the origins of cheating in early game development and its evolution into a global underground economy. It explores how cheat tools emerged, adapted to online play, and became increasingly commercialized, as well as how anti-cheat systems evolved in response.
- **Chapter 4: The Prevalence and Impact of Cheating in Online Multiplayer Games** – Evaluates the real-world effects of cheating on player experience and game ecosystems. It includes a case study on Counter-Strike 2, analyzing VAC ban data, reporting limitations, psychological and economic effects, and community perception.
- **Chapter 5: Injection and Detection Techniques** – Provides a technical overview of how cheats inject code into game processes, and the corresponding detection and evasion techniques used. It also reviews Valve's anti-cheat technologies and their limitations.
- **Chapter 6: Internal Game Structure: Logic Behind Counter-Strike 2** – Describes the architecture of the game engine, focusing on entity list management, memory structure, and reverse engineering techniques to understand how data is stored and manipulated internally.
- **Chapter 7: Cheat Features Implemented in Our Cheat for CS2** – Details the development of specific cheating features, including Chams for visibility, Aimbot systems, and bone rendering for player models. It includes implementation details such as function hooking, mathematical computations, and stealth mechanisms.
- **Bibliography** – Compiles the academic literature, technical documentation, and community-based research that informed the thesis.

---

# 2. FROM PRIMITIVE SURVIVAL TO DIGITAL DOMINATION

## 1 The Competitive Nature of Humanity

Throughout history, humanity has been shaped by an innate drive for **superiority, dominance, and recognition**. From primitive survival instincts to modern technological achievements, the desire to **outperform others** has fueled progress in every field—science, warfare, politics, sports, and, in more recent times, digital environments.

This competitive nature is not merely a byproduct of ambition but a **deeply ingrained psychological mechanism** that has guided human behavior for centuries. Understanding the psychological foundations of this drive is crucial to explaining its persistence and evolution over time. Festinger [*1*], through his **Social Comparison Theory**, posited that individuals have an inherent tendency to evaluate themselves in relation to others. This constant self-assessment fosters competition as people seek to validate their abilities and secure a sense of superiority. Furthermore, McClelland [*2*], in his **Achievement Motivation Theory**, demonstrated that the drive to set goals and surpass one's peers is not simply a social construct but an **intrinsic component of human motivation**, crucial for both personal fulfillment and status recognition. These psychological insights reveal why competition remains a defining force in human interactions, shaping not only individual aspirations but also broader societal structures.

From an **evolutionary perspective**, competition has been a fundamental survival mechanism, shaping the trajectory of human development. The earliest human societies relied on **strategic thinking and adaptability** to outcompete rivals in hunting, warfare, and resource acquisition. As civilization advanced, this drive extended beyond mere survival, influencing **cultural, intellectual, and economic structures**. Even from childhood, individuals naturally seek to establish their competence relative to others, reinforcing competition as a deeply rooted cognitive and social process.

**Thus, what began as a necessity for survival has evolved into a complex, multifaceted force that continues to define human ambition and progress.**

With the rise of modern technology, **digital environments** have emerged as a new arena for competition. The invention of **computers**, followed by the creation of the **internet**, revolutionized how humans interact, process information, and engage in competitive activities. The transition from early computational machines to networked systems paved the way for **video games**, where structured digital challenges allow indi-

viduals to test their abilities against both artificial intelligence and human opponents. These digital platforms tap into the same fundamental competitive instincts seen in traditional settings, providing an environment where players can measure their skills, engage in **goal-oriented behavior**, and gain recognition within their respective communities. This further reinforces the notion that competition, regardless of its medium, remains a **powerful driver of human engagement and achievement**.

As in other competitive domains, players in video games seek to **optimize their performance**, discover hidden mechanics, and exploit advantageous strategies. Research on competitive behavior suggests that when individuals encounter barriers to success, they often look for alternative ways to improve their standing [*3*]. This phenomenon is particularly pronounced in online multiplayer games, where **leaderboards, ranking systems, and social validation** create high levels of pressure to excel. Whether through **pure skill, strategic knowledge, or leveraging unintended mechanics**, players are constantly seeking an edge over their opponents.

To fully understand how digital competition has evolved into modern gaming culture, it is essential to examine the **historical progression** that led to this moment. The following chapters will explore the origins of **computation**, the development of **networked systems**, and the emergence of **video games** as a dominant form of entertainment. By tracing this **technological evolution**, we will gain insight into how human competition has been redefined in the digital age, leading to the creation of **virtual battlefields** where **skill, innovation, and adaptability** determine success.

# 2 The Birth of Computation and the Internet: A New Digital Battleground

The journey toward digital competition began with humanity's earliest attempts to **automate reasoning and computation**. Since antiquity, civilizations have sought ways to enhance their capacity for **mathematical problem-solving**, recognizing its crucial role in **commerce, astronomy, and engineering**. As societies advanced, so did their tools, gradually evolving from rudimentary counting mechanisms to sophisticated machines capable of complex calculations.

## 2.1 Early Computational Tools: From the Abacus to Mechanical Calculators

To facilitate numerical tasks, ancient civilizations developed devices that enhanced computational accuracy and efficiency. Among the earliest was the **Abacus** (circa 2400 BCE), widely used across Mesopotamia, China, and later Europe. By allowing merchants and scholars to perform arithmetic operations swiftly, this tool laid the foundation for mechanical computation.

By the 17th century, advancements in mechanical engineering led to more sophisticated devices, such as **Pascal's Adding Machine** (1642). Invented by **Blaise Pascal**, this early mechanical calculator used a system of rotating gears to perform basic arithmetic operations, marking a significant step toward reducing cognitive labor in mathematical tasks. Such innovations foreshadowed the emergence of machines that could not only compute but also **execute logical operations**.

## 2.2 The 19th-Century Revolution: The Birth of Programmable Machines

Despite these early developments, computation remained a largely manual process until the 19th century, when the concept of **programmable machines** emerged. The most influential figure in this transition was **Charles Babbage**, who designed the **Analytical Engine**—a theoretical device that introduced fundamental computing principles such as **conditional logic, input/output processing, and memory storage**. Although never constructed in his lifetime, this innovation established the conceptual groundwork for modern computing.

Building upon Babbage's vision, **Ada Lovelace** recognized that such machines could transcend mere arithmetic calculations. Her development of the first algorithm for the Analytical Engine positioned her as the world's first programmer. More importantly, her insights into computational logic and symbolic processing laid the foundation for **modern programming languages**, demonstrating that machines could be programmed to execute intricate sequences of instructions.

## 2.3 The Acceleration of Computation in the 20th Century

The **20th century** marked a turning point in computational development, driven by both **military necessity and scientific ambition**. The transition from mechanical calculators to fully electronic computers revolutionized data processing, transforming computation from a supportive tool into an essential driver of technological progress.

### 2.3.1 World War II and the Role of Computation

One of the most significant catalysts for computing advancements was **World War II**, where rapid and precise calculations were required for **cryptographic warfare, ballistic trajectory analysis, and logistical coordination**. During this period, **Alan Turing** formalized the concept of **algorithmic processing** through his **Turing Machine**, providing the theoretical model for modern computers. His contributions to cryptanalysis, particularly in breaking the **Enigma** cipher, underscored the strategic importance of computational power.

Parallel to Turing's work, large-scale electronic computers were developed to support wartime efforts. The British-built **Colossus** (1943), one of the first programmable elec-

tronic computers, was instrumental in deciphering enemy communications. Meanwhile, in the United States, the creation of **ENIAC** (1945) enabled rapid ballistic calculations, enhancing military planning and precision. These advancements established computation as a powerful tool for **automated problem-solving**, demonstrating its potential beyond manual calculations.

### 2.3.2 Post-War Expansion: From Military to Commercial Computing

Following the war, computing technology expanded into **academia, industry, and eventually, everyday life**. The late 1940s and early 1950s saw the rise of **mainframes**, large-scale machines used by governments and research institutions for data processing, scientific simulations, and business applications. This period also witnessed the development of the **transistor** (1947), which replaced vacuum tubes, making computers smaller, more efficient, and commercially viable.

The transition toward personal computing accelerated with the advent of **integrated circuits** in the 1960s, leading to the emergence of **personal computers (PCs)** in the 1970s and 1980s. Companies such as **IBM**, **Apple**, and **Microsoft** played a pivotal role in bringing computing to homes and workplaces. The introduction of graphical user interfaces (GUIs), exemplified by the **Apple Macintosh** (1984), transformed computers from specialized industrial tools into **everyday consumer devices**.

## 2.4 The Birth of the Internet: From Military Networks to Global Infrastructure

While computing power had grown exponentially, its true transformation came with the advent of **networked computing**. The first major step in this direction was the creation of **ARPANET** in the late 1960s, a project funded by the U.S. Department of Defense. Designed as a **resilient, decentralized communication network**, ARPANET introduced **packet-switching**, a method of transmitting data in independent packets that improved reliability and efficiency.

In the early 1980s, the standardization of **TCP/IP protocols** allowed different networks to communicate seamlessly, marking the birth of the modern **Internet**. The introduction of the **Domain Name System (DNS)** in 1984 simplified navigation, replacing numerical IP addresses with human-readable domain names. However, the most transformative moment came in 1991 with the development of the **World Wide Web** by **Tim Berners-Lee**, which enabled hyperlinked browsing and revolutionized digital interaction.

By the mid-1990s, the internet had transitioned from a military and academic tool to a **global infrastructure**, fueled by the proliferation of personal computers and the expansion of commercial internet service providers (ISPs). This transformation redefined human interaction, enabling not only global communication but also the rise of **competitive digital spaces**.

## 2.5 The Digital Arena: From Computing to Competition

Before the era of global matchmaking, however, early competitive gaming took root in informal spaces like cybercafés and home LAN parties. These grassroots gatherings laid the cultural foundation for modern esports. Figure 2.1 captures one such moment.

Figure 2.1: Late 1990s LAN Party.

As computing evolved into an interconnected ecosystem, competition within digital environments intensified. No longer confined by physical constraints, individuals and organizations began to seek new ways to **optimize systems, manipulate algorithms, and dominate digital spaces**. Hackers, programmers, and early adopters of networked technologies engaged in skill-based challenges—exploring vulnerabilities, exploiting loopholes, and testing the boundaries of online security and performance.

Simultaneously, the rise of online connectivity redefined one of computing's most engaging applications: **video gaming**. What was once a solitary or local experience evolved into a **global competitive platform** where players could measure their skill against opponents worldwide. The emergence of online multiplayer in the late 1990s and early 2000s marked a pivotal shift, transforming games from recreational pastimes into structured, high-stakes competitions.

This transition was powered by several key technological advancements. The proliferation of broadband internet enabled real-time multiplayer interactions with reduced latency, fostering smoother and more dynamic gameplay. Matchmaking algorithms began pairing players based on skill levels, creating balanced and engaging encounters.

Persistent ranking systems and leaderboards added layers of progression, status, and social prestige—encouraging players not just to play, but to win.

## 2.6 From Networks to Virtual Battlefields

The increasing complexity of networked systems created digital battlegrounds where individuals and teams competed for dominance. Online gaming communities flourished, with players dedicating time to mastering mechanics, developing strategies, and optimizing their performance. Titles such as **Counter-Strike**, **StarCraft**, and **Quake** set early standards for competitive gaming, emphasizing skill, reflexes, and tactical awareness.

Unlike other digital challenges, video games provided **transparent rules, structured rankings, and quantifiable performance metrics**. These elements made gaming one of the most accessible forms of competitive engagement, allowing players worldwide to test their abilities on a level playing field. The introduction of dedicated esports tournaments and professional leagues further legitimized gaming as a form of structured competition, attracting sponsorships, media attention, and substantial prize pools.

The rise of **competitive gaming** was not merely an evolution of digital entertainment—it was a natural extension of human competition into the virtual realm. What began as casual online matches evolved into **organized tournaments, professional leagues, and eSports**, where players compete for prestige, financial rewards, and international recognition.

As technology continues to redefine the nature of digital interaction, gaming stands as one of the most structured and influential arenas of online competition. The following chapters will explore the evolution of **competitive gaming**, tracing its origins from early arcade tournaments to its establishment as a global entertainment and professional industry.

---

# 3. THE EVOLUTION OF CHEATING: FROM DEBUGGING TOOLS TO AN ORGANIZED INDUSTRY

The history of cheating in video games is deeply intertwined with the evolution of the industry itself. What began as debugging tools for developers gradually transformed into a clandestine culture of modifications, and with the advent of online gaming, an organized and profitable industry with ties to cybercrime [*4*]. Simultaneously, the fight against cheating led to the development of **anti-cheat systems**, which evolved from simple integrity checks to sophisticated solutions incorporating artificial intelligence and deep system access [*5*].

## 1 The Early Days: Debugging Tools and Experimental Modifications

Before video games became a mainstream form of entertainment, early cheating emerged as a necessity within software development. In the 1970s and 1980s, developers embedded debugging tools within their games to facilitate testing. These systems allowed programmers to modify internal values such as lives, ammunition, and speed, enabling them to test different scenarios without playing through the entire game [*6*].

Some of these tools remained accessible in commercial releases, either unintentionally or as a deliberate choice by developers. This led to the creation of the first **cheat codes**, which were key combinations or commands that altered gameplay. One of the most iconic examples is the **Konami Code** (↑ ↑ ↓ ↓ ← → ← → B A), introduced in *Gradius* (1986) and later popularized in *Contra* (1988) [*7*]. While these codes were not designed to affect competition, they set a precedent for the intentional modification of gameplay.

## 2 The Rise of Cheating in Consoles and PC

As the video game industry grew throughout the 1980s and 1990s, cheating expanded beyond debugging tools and built-in cheat codes. With the advent of home consoles, external devices specifically designed to modify gameplay began to appear.

## 2.1 Cheat Devices: Game Genie, GameShark, and Pro Action Replay

The first dedicated cheat devices were **modification cartridges**, such as the **Game Genie** (1990) and the **GameShark** (1995). These hardware add-ons allowed players to inject custom modifications into a game's memory, altering key parameters such as health, ammunition, and physics [***8***]. While initially marketed as harmless tools for enhancing single-player experiences, they introduced the concept of **real-time game manipulation**, which would later be adapted for online environments.

On the PC side, the introduction of **memory editors** provided users with a more flexible and powerful way to modify game values at runtime. Early programs like **Cheat Engine** (2000) enabled players to search for specific in-game variables and alter them directly, bypassing the limitations of built-in cheat codes [***9***]. This shift from **developer-sanctioned cheats** to **unauthorized external modifications** laid the foundation for more advanced game-hacking techniques.

As gaming technology evolved, so did the tools available for modifying game memory structures. More sophisticated programs such as **ReClass.NET** emerged, allowing users to analyze and modify game memory at a structural level. Unlike traditional memory scanners, ReClass.NET enables users to reverse-engineer game memory layouts by dynamically inspecting class structures and data offsets in real-time [***10***]. This advancement significantly improved the ability of cheat developers to create complex modifications, from automatic aimbots to more intricate exploits that manipulate how game logic processes data.


Figure 3.1: Game Genie

These tools, originally designed for educational and debugging purposes, soon became widely used in cheat development, particularly in online gaming environments. Their increasing complexity meant that anti-cheat systems had to adapt rapidly, leading to an ongoing technological arms race between cheat developers and security teams.

# 3 The Transition to Online Gaming: Cheating Becomes a Competitive Threat

The rise of online multiplayer gaming in the late 1990s and early 2000s fundamentally changed the impact of cheating. Unlike single-player cheats, which only affected the individual experience, **online cheating directly disrupted fair competition** [***5***]. As games like **Quake** (1996), **Counter-Strike** (1999), and **Diablo II** (2000) introduced matchmaking and ranking systems, the incentive to cheat increased dramatically [***11***].

The growing importance of competitive integrity led developers to take more aggressive measures against cheating, resulting in the creation of **server-side and client-side anti-cheat mechanisms**. However, as anti-cheat solutions became more sophisticated, so too did cheating techniques, leading to an ongoing technological arms race between game security teams and cheat developers.

This shift led to the development of the first organized cheat communities, where players and programmers shared exploits, scripts, and modified game clients. It also marked the beginning of an **arms race** between cheat developers and game security teams, as new forms of cheating emerged alongside increasingly complex anti-cheat measures.

# 4 The Birth and Evolution of Anti-Cheat Systems

As online gaming became increasingly competitive, the integrity of fair play was challenged by the widespread use of cheats. Unlike single-player modifications, which had no broader consequences, cheating in multiplayer environments compromised **matchmaking, rankings, and esports competitions**, creating frustration among players and damaging the credibility of competitive gaming.

Early attempts to mitigate cheating relied on **server-side validation**, where game servers monitored player actions for anomalies that suggested external manipulation. These checks aimed to detect unnatural behavior, such as **impossible movement speeds, improbable accuracy, and modifications to game assets** that altered visibility constraints. However, this approach had significant limitations, as many cheats operated entirely on the **client side**, modifying game data before it was transmitted to the server. As a result, more sophisticated **client-side anti-cheat solutions** were developed to monitor and detect unauthorized modifications directly within the player's system.

One of the first dedicated anti-cheat programs was **PunkBuster**, developed by Even Balance, Inc. and introduced in **2000**. Initially implemented in *Return to Castle Wolfenstein* and later in *Battlefield 1942*, PunkBuster marked a shift from basic server-side detection to **active client-side monitoring**. It worked by scanning a player's system for **known cheat signatures**, verifying **game file integrity**, and capturing periodic **in-game screenshots** to identify visual modifications such as **wallhacks**. Additionally, PunkBuster maintained a **global ban list**, preventing known cheaters from accessing protected servers. Despite these advancements, **cheat developers quickly adapted** by employing techniques such as **code obfuscation**, allowing their software to remain undetected. PunkBuster also faced criticism for its **aggressive scanning approach**, which occasionally resulted in **false positives**, banning legitimate players due to conflicts with background applications.

In **2002**, Valve introduced **Valve Anti-Cheat (VAC)** alongside *Counter-Strike 1.6*, implementing a more **automated and data-driven approach** to cheat detection. Unlike PunkBuster, which **immediately expelled** suspected cheaters, VAC introduced a **delayed banning system**. This strategy allowed suspected cheaters to continue playing while logging their infractions, making it harder for cheat developers to **identify detection triggers** and modify their cheats accordingly. In addition to **signature-based detection**, VAC utilized **heuristic analysis** to recognize **abnormal behavioral patterns**, refining its detection algorithms over time. By integrating **server-side validation** with **machine learning techniques**, VAC continuously improved its ability to detect and counteract evolving cheats. However, since it **operated at the user level** rather than the **kernel level**, advanced cheats that manipulated system memory with higher privileges could still **bypass its protections**.

As **cheat developers refined their evasion techniques**, more **aggressive anti-cheat solutions** emerged. One of the most notable advancements was **BattleEye**, introduced in **2004** and later adopted in games like *Arma 2*, *Rainbow Six Siege*, and *PUBG*. Unlike VAC, which prioritized **passive detection**, BattleEye enforced **stricter security measures** by running with **elevated system privileges**. By operating at the **kernel level**, it could **detect and block cheat software** that manipulated **game memory at a deeper level**, making it significantly harder for external programs to **alter in-game data undetected**. This approach allowed BattleEye to **intercept and prevent unauthorized modifications** before they could affect gameplay, offering a **more proactive defense** against sophisticated cheats.

As **competitive gaming and esports** continued to grow, **Riot Games** introduced a new anti-cheat solution, **Vanguard**, designed specifically for *Valorant*. Released in **2020**, Vanguard took **anti-cheat enforcement to an unprecedented level** by implementing **persistent kernel-level monitoring**, meaning it **actively ran as a background process from the moment the operating system started**. This deep system integration enabled it to **detect and prevent unauthorized modifications** at the **lowest levels of system memory**, making it one of the **most invasive yet effective anti-cheat**

**solutions**. However, its **intrusive nature sparked controversy** among players concerned about **privacy**, as the software had **continuous access to system processes even when the game was not running**.

The **evolution of anti-cheat systems** reflects an **ongoing technological arms race** between **cheat developers and game security teams**. While early efforts relied on **rudimentary server-side validation**, modern solutions integrate **behavioral analysis, machine learning, and kernel-level access** to provide more robust protection. Despite these advancements, the constant adaptation of cheat developers ensures that no anti-cheat system remains **impenetrable for long**, perpetuating a **continuous cycle of exploitation and countermeasures** in the digital battlefield of competitive gaming.

This historical evolution has given rise to a wide variety of anti-cheat implementations, each shaped by the needs and challenges of its era.

The continuous development of anti-cheat solutions over the past two decades can be summarized through a timeline of key technologies. Figure 3.2 provides a chronological overview of the most influential anti-cheat systems, from early client-side detectors to modern kernel-level and behavior-based solutions.

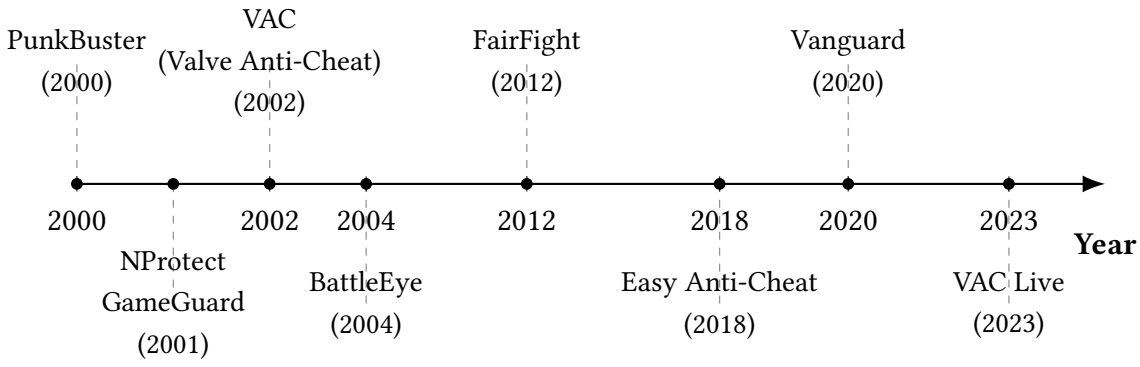


Figure 3.2: Chronological overview of key anti-cheat systems introduced between 2000 and 2023.

Among the systems depicted in the timeline, several stand out for their unique approach and impact.

**nProtect GameGuard**, launched in 2001, was one of the first anti-cheats to adopt rootkit-like behavior. Widely used in Asian MMORPGs, it embedded itself deeply in the operating system to prevent cheats from interacting with game processes, although this also raised concerns about privacy and system stability.

**FairFight**, introduced in 2012, departed from traditional detection mechanisms by using server-side statistical analysis to flag abnormal player behavior. Rather than inspecting memory or processes, it relied on gameplay metrics such as accuracy, reaction time, and movement patterns—making it less intrusive, but also vulnerable to both false positives and high-skill players.

**Easy Anti-Cheat (EAC)**, which gained traction in 2018, became a popular choice for fast-paced competitive games. It combines kernel-level access with performance-friendly design and supports real-time detection of unauthorized drivers, injected DLLs, and tampering attempts—balancing robustness and user transparency.

Finally, **VAC Live**, unveiled in 2023, represents Valve's shift toward real-time, data-driven cheat prevention. Unlike the original VAC system, which relied on delayed bans, VAC Live aims to detect and react to cheats during gameplay, making it a critical component in safeguarding the integrity of high-stakes esports matches.

# 5 The Commercialization of Cheating

The rapid commercialization of cheating has transformed it from a niche activity into a highly organized industry, offering both software-based and hardware-level exploits. Specialized forums such as **UnKnoWnCheaTs**, **MPGH**, and **ElitePVPers** have become central hubs for distributing cheats, providing resources for reverse engineering, memory manipulation, and anti-cheat evasion. These platforms, along with encrypted messaging services like **Discord** and **Telegram**, have facilitated the sale and distribution of sophisticated cheats through subscription models and private communities.

Table 3.1 summarizes some of the most prominent cheat providers currently active in the CS2 ecosystem.

| Provider | Released | Subscription Price | Access Model |
|---|---|---|---|
| **Neverlose** [*12*] | 2020 | €17.10/month, €44.10/3 months, €134.10/year | Invite-only |
| **Memesense** [*13*] | 2023 | $4/14 days, $8/month, $12/3 months | Public access |
| **Fatality.win** [*14*] | 2022 | Premium pricing (variable) | Invite-only |
| **Project Infinity** [*15*] | 2017 | €14.99/month, €29.99/3 months, €99.99 lifetime | Public access |
| **Midnight** [*16*] | 2019 | £5.42/month | Public access |
| **Onetap** [*17*] | 2024 | Pricing not publicly disclosed | Public access |
| **Skeet (Gamesense)** [*18*] | 2016 | $16/month (via tokens, fluctuating) | Invite-only + HWID locked |

Table 3.1: Overview of major CS2 cheat providers, including release year, pricing models, and access restrictions.

As cheat developers adopt increasingly advanced techniques, such as kernel-level drivers and DMA-based exploits, anti-cheat systems are forced into a continuous state of adaptation. The emergence of software-based cheats, particularly those using DLL injection, code hooking, and API interception, has proven to be one of the most adaptable and challenging threats. These methods allow cheat developers to modify game functions dynamically, bypassing traditional detection systems.

## From History to Implementation: A Technical Transition

Having traced the historical and sociotechnical evolution of cheating—from its roots in debugging tools to its current status as a commercialized, clandestine industry—we now shift focus toward the technical foundations that sustain it today.

The next chapters will abandon the literary and historical perspective in favor of a rigorous, low-level examination of how modern software-based cheats are constructed and deployed. We will explore the inner mechanics of code injection, memory manipulation, and anti-cheat evasion techniques, providing a detailed view of the technological arms race from a reverse engineering and systems programming standpoint.

With the historical context established, it is time to dissect the machinery.

# 4. THE PREVALENCE AND IMPACT OF CHEATING IN ONLINE MULTIPLAYER GAMES

The integrity of online multiplayer games continues to be seriously compromised by the widespread use of cheats. This practice not only undermines fair play but also has a profound impact on player satisfaction and long-term engagement. In this chapter, we explore the extent of the cheating phenomenon, focusing particularly on *Counter-Strike 2* (CS2), and argue that cheating constitutes one of the most significant threats to the sustainability of competitive online ecosystems. As the popularity of online titles grows, particularly those with competitive rankings and in-game economies, the incentives to cheat grow stronger, creating a persistent and evolving threat.

## 1 Cheating as a Critical Threat to Player Retention

Cheating is not a marginal problem—it is a critical factor driving user attrition. According to a global report by Irdeto, 60% of players stated that cheating negatively impacted their multiplayer experiences **19**. More alarmingly, 77% of respondents indicated they would abandon a game if cheating persisted. These figures illustrate not only the widespread perception of unfairness but also the economic risk for developers, as high player churn can significantly affect active user metrics and revenue from in-game purchases.

Table 4.1: Survey Results on Cheating in Online Games (Irdeto, 2018)

| **Survey Statement** | **Percentage of Respondents** |
|---|---|
| Experienced negative impact due to cheating | 60% |
| Would stop playing a game because of cheaters | 77% |
| Support permanent bans for cheaters | 69% |
| Believe developers are not doing enough | 55% |

These statistics highlight the severity of the problem: cheating not only degrades the moment-to-moment experience but also leads to long-term disengagement from the game. For games that rely on consistent engagement, particularly in esports ecosystems, this presents a fundamental challenge.

# 2 Case Study: Cheating in *Counter-Strike 2*

## 2.1 Player Base and VAC Ban Statistics

*Counter-Strike 2* (CS2) continues to be a dominant force in the competitive first-person shooter (FPS) genre. In March 2025, the game recorded a peak of over 1.8 million concurrent players, capping off more than a year of steady growth. Notably, this trend began in early 2024: January saw over 760,000 average players and a peak of 1.27 million, followed by a significant increase in February, with averages rising to over 786,000 and peaks surpassing 1.34 million.

This upward trajectory reached new highs in March 2025, reaffirming CS2's popularity and reinforcing the importance of maintaining competitive integrity. As the player base grows—especially among professional players, content creators, and tournament organizers—the role of Valve's anti-cheat measures becomes increasingly critical.

Figure 4.1 visualizes the evolution of the CS2 player base from January 2024 to March 2025, including both average and peak monthly player counts based on publicly available statistics.


Figure 4.1: Counter-Strike 2 player count evolution from January 2024 to March 2025.

While these figures indicate proactive enforcement, they also reveal the limitations of current anti-cheat mechanisms. Many cheaters continue to operate undetected for extended periods, often using private or custom-developed cheats that evade signature-based detection. Furthermore, the widespread use of smurfing—creating multiple low-profile accounts to circumvent bans—diminishes the long-term effectiveness of VAC.

Another critical concern is the detection delay: cheats frequently remain effective for days or even weeks before countermeasures are deployed. This window allows cheaters to impact competitive integrity, frustrate legitimate players, and erode trust in the matchmaking system. According to a recent analysis, the average delay between a cheater's last match and their eventual VAC ban was around 20 days, with only 28% of bans occurring within 24 hours **20**. During this time, banned players had already participated in over 32,000 matches, with most games featuring only a single cheater. This confirms that cheaters actively participate in regular gameplay rather than confined abuse scenarios like case farming, severely degrading the experience for legitimate players.

Table 4.2: Activity of Banned Accounts Prior to Enforcement (Jan–Apr 2024)

| **Metric** | **Estimated Value** |
|---|---|
| Average delay between last match and VAC ban | 20 days |
| Percentage of bans issued within 24 hours | 28% |
| Total matches played by banned accounts | 32,000+ |
| Percentage of matches with only one banned player | 97% |
| Evidence of case farming behavior | Not significant |
| Gameplay pattern analysis | Consistent with competitive play |

These statistics highlight not only the persistence of the cheating problem but also the urgent need for more advanced, adaptive anti-cheat technologies that prioritize early detection and real-time behavioral analysis.

## 2.2 The Reporting System and Trust Factor Limitations

CS2 incorporates a system known as the *Trust Factor*, an invisible matchmaking metric introduced by Valve in late 2017 to improve the quality and fairness of online matches. Unlike traditional ranking systems, which focus primarily on in-game performance (e.g., win/loss ratio, kill/death ratio), the Trust Factor seeks to evaluate the overall behavior and reputation of a player across the Steam ecosystem.

Although Valve has not publicly disclosed the full algorithmic details of how Trust Factor is calculated, community research, developer hints, and user experimentation have led to the identification of several factors believed to influence the Trust Factor score:

- **VAC bans and Overwatch bans:** Any history of cheating or toxic behavior significantly reduces Trust Factor.

- **Game-specific reports:** Frequency and validity of reports for cheating, griefing, or abusive conduct.
- **Commendations:** Positive in-game feedback from teammates (e.g., ”Friendly”, ”Good Leader”) is assumed to improve Trust Factor.
- **Steam account age:** Older accounts are generally considered more trustworthy than newly created ones.
- **Community participation:** Use of features like Steam Groups, Reviews, and profile visibility.
- **Hardware/software flags:** Suspicious configurations or third-party software may trigger internal risk indicators.
- **Behavior across multiple games:** Toxic behavior or bans in other Valve titles can influence Trust Factor globally.
- **Friend network quality:** Associations with banned or low-Trust players may negatively impact your own score.

Players with lower Trust Factor scores—often due to false reports, negative behavior, or even external factors—can find themselves matched with suspected cheaters or toxic teammates. The system does not communicate its internal logic to players, which contributes to confusion and a perception of arbitrariness.

Furthermore, the reporting system itself is vulnerable to abuse. Coordinated reporting by enemy players or even automated bots—known as *report bots*—can unfairly penalize innocent users. These bots are sold as a service on underground forums and marketplaces, enabling malicious actors to artificially lower another player’s Trust Factor through mass false reports. Victims of such actions may find themselves trapped in low-quality matchmaking pools with increased exposure to cheaters.

Conversely, a market has emerged for *commend bots*, which artificially inflate Trust Factor scores by using botted accounts to issue in-game commendations. These commendations, such as ”Good Leader”, ”Friendly”, or ”Teacher”, are believed to be positively weighted in Trust Factor calculations. These services exploit the opacity of Trust Factor metrics and undermine the intended spirit of fair matchmaking by allowing players to manipulate their standing through paid influence.

Consequently, while the Trust Factor system aims to protect competitive integrity, its opacity and susceptibility to manipulation can exacerbate the issues it was designed to mitigate. Calls for greater transparency, audit trails, and player-facing Trust metrics have been common within the community, especially among long-term and high-investment users.

## 2.3 Economic and Psychological Impact

Valve's ban waves have economic repercussions as well. In early 2025, a VAC wave eliminated accounts with over two million USD in inventory value **21**. This not only disrupted the secondary skin market but also created anxiety among legitimate players. Many users invest both time and money into building inventories, and the fear of false positives or lack of transparency in ban decisions can alienate loyal players.

The psychological burden is also considerable: legitimate users who are falsely flagged or who perceive the system as ineffective may lose confidence in the platform, resulting in churn. In ranked systems, where competition is meant to be meritocratic, repeated exposure to cheating can erode motivation and participation. For new players, early exposure to blatant unfairness can lead to immediate disengagement.

Moreover, the persistent presence of cheaters devalues competitive ranks and tournaments, potentially deterring sponsors and professional teams from engaging with the title long-term. In such an environment, even minor perceived imbalance can be seen as systemic failure, amplifying negative sentiment across the community.

# 3 Community Perception and Player Sentiment

The response from the gaming community is often one of frustration, skepticism, and fatigue. On platforms like Reddit, Steam, and YouTube, countless posts and videos chronicle blatant cheaters in ranked games, sometimes persisting for weeks without enforcement. These examples accumulate into a narrative of neglect, regardless of the actual efforts made by developers.

Professional players and streamers have also spoken out, emphasizing how recurring cheating incidents damage both their gameplay experience and viewer engagement. For example, repeated matches against obvious cheaters during live broadcasts diminish entertainment value and create reputational risks for the publisher.

This perception of systemic vulnerability further undermines confidence in anti-cheat frameworks and leads to a normalization of defeatism among legitimate players. Forums often reflect sentiments such as "everyone is cheating at higher ranks" or "VAC does nothing," which signal the erosion of perceived fairness.

In a competitive environment where rank integrity is paramount, even a minority of cheaters can poison the perception of fairness. This phenomenon fuels resignation and, eventually, player abandonment. A toxic perception climate can spread faster than actual cheating prevalence, compounding the damage.

## 4 Legal and Ethical Implications of Cheating Software

Beyond technical responses, several companies have turned to legal action. The production and distribution of cheat software violates terms of service and, in some jurisdictions, constitutes intellectual property infringement or even fraud.

Bungie and Ubisoft, for example, have successfully sued cheat developers for their role in undermining game integrity and profiting from illicit tools. Notably, in 2023, Bungie won a $12 million lawsuit against cheat seller AimJunkies for its role in distributing software that compromised *Destiny 2*'s ecosystem. Other cases have followed in Australia, Germany, and the United States, with courts increasingly recognizing the damages associated with commercial cheating operations.

These legal precedents are critical not only for deterrence but also to assert the enforceability of end-user license agreements (EULAs) in gaming contexts. They signal that companies are increasingly willing to pursue judicial avenues when technical means fall short. Legal actions also help target the source of advanced cheat development, limiting their distribution and profitability.

From an ethical perspective, the existence of pay-to-win cheats challenges the spirit of competitive gaming. When fairness is monetized and subverted, the entire premise of esports and ranked systems is put into question, leading to long-term damage to brand value and consumer trust.

## 5 Anti-Cheat Technologies and Their Limitations

While developers continue to enhance their anti-cheat frameworks, existing solutions often operate reactively. Valve's VAC system relies on detecting known cheat signatures. Although the update to VacNet 3.0 brings improvements in behavioral analysis and detection latency, many cheats remain undetected due to obfuscation, polymorphism, and manual injection methods.

Furthermore, VAC does not operate at the kernel level, leaving a gap exploited by more sophisticated cheating tools. Unlike Riot's Vanguard, which implements invasive low-level monitoring, VAC opts for user privacy—but at the cost of reduced enforcement power. This trade-off reflects a broader industry dilemma between effectiveness and user rights.

More innovative systems, such as AI-driven cheat behavior modeling or client-side encryption, are being tested, but none have yet offered a comprehensive solution. Cheat developers adapt quickly, often using anti-debugging and virtualization evasion techniques, requiring anti-cheat providers to continually evolve in a costly and time-sensitive arms race.

Cheating poses a systemic threat to the health and sustainability of online multiplayer games. The data reveals that while anti-cheat systems like VAC are essential, they are not enough to eliminate the problem. The high number of bans, combined with persistent community frustration and the economic damage inflicted, indicate that many cheaters continue to escape detection.

To restore trust and ensure fair play, the industry must explore more proactive, transparent, and adaptive anti-cheat solutions, supported by legal frameworks and community-driven oversight. Educational initiatives, transparency reports, real-time user flagging tools, and tighter platform-level enforcement (e.g., via Steam or Xbox Live) could help close current enforcement gaps. Without such evolution, even the most successful titles risk losing their competitive legitimacy and alienating their player base.

---

# 5. INJECTION AND DETECTION TECHNIQUES

## 1 Why Injection Matters in Cheat Development

In the realm of professional game cheating, nearly all serious and commercially available cheats—especially those developed for competitive titles like *Counter-Strike 2*—are built as **DLLs** that are injected into the game process. This design choice is not incidental; it is fundamental to how modern cheats operate and achieve their functionality.

Injecting a DLL into the game allows the cheat to execute in **runtime** alongside the game itself. This real-time coexistence grants the cheat direct access to the game's memory, logic, and rendering pipeline. As a result, injected cheats can:

- **Hook internal functions** to intercept or modify game behavior (e.g., aim mechanics, visibility checks).
- **Modify memory values** to influence in-game parameters (e.g., recoil, ammunition, speed).
- **Read game data structures** to extract critical information (e.g., player positions, health, weapons).
- **Render overlays** by drawing directly on top of the game window (e.g., ESP, radar, crosshairs).
- **Automate gameplay** with logic that integrates seamlessly with the game's update loop.

This deep integration is only possible through in-process code execution, which is why DLL injection is the industry standard in cheat development. Accordingly, anti-cheat systems like **Valve Anti-Cheat (VAC)** place significant emphasis on detecting and blocking injection attempts, making injection both the cornerstone of modern cheating and the primary target for detection.

The following sections explore the most relevant injection techniques used in game cheating and the methods employed by anti-cheat systems to detect and prevent them. The objective is to provide a conceptual overview without delving into low-level implementation details.

# 2 Overview of Code Injection Techniques

Code injection refers to the practice of inserting arbitrary executable code into a running process. In the context of cheating, this typically involves loading a custom Dynamic Link Library (DLL) or executing shellcode within the game's memory space to gain persistent and privileged access to game internals.

## Traditional Methods

Early cheats commonly relied on standard Windows functions such as:

- `LoadLibrary`: Loads a DLL into the target process.
- `CreateRemoteThread`: Spawns a new thread in another process to execute injected code.

While these approaches are simple and effective in general software development, they are highly detectable. Anti-cheat systems like VAC monitor API usage, track module registrations, and flag suspicious thread creation patterns. As a result, traditional injection methods are now largely ineffective for persistent, undetected cheating.

## Manual Mapping

**Manual Mapping** has become the preferred method for injecting cheats into modern games. Rather than relying on the Windows loader, this technique manually replicates the steps involved in loading a DLL, entirely within user space.

Manual Mapping is highly effective because:

- It bypasses standard API calls like `LoadLibrary`, avoiding direct detection.
- Injected modules do not appear in the target process's module list (PEB).
- Execution can be achieved through stealthy mechanisms such as thread hijacking or shellcode trampolines.

Due to its stealth and flexibility, Manual Mapping is considered the most robust and professional method in the current landscape of cheat development.

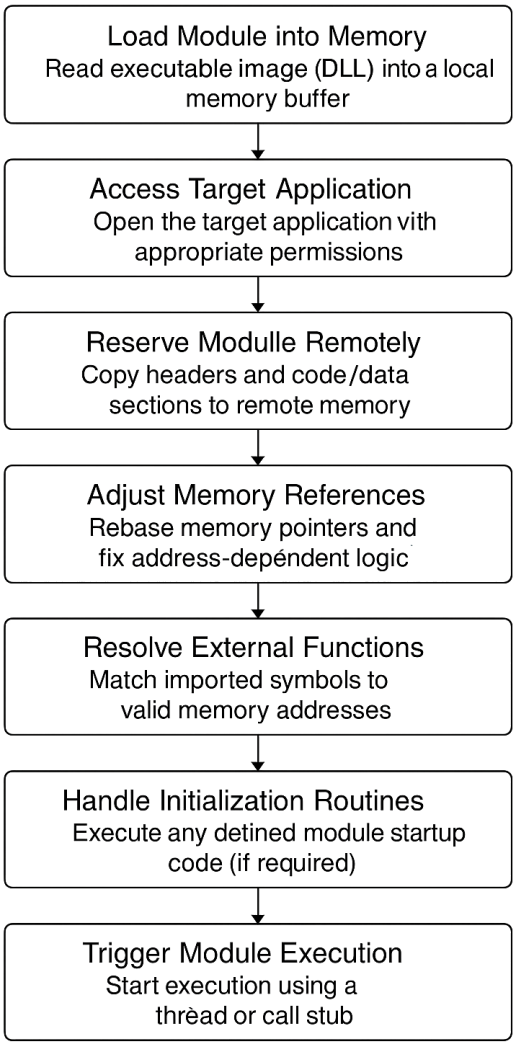


Figure 5.1: Abstracted flowchart of dynamic module injection.

# 3 Detection Techniques in Anti-Cheat Systems

Anti-cheat engines like VAC continuously scan game processes for indicators of unauthorized code. Their goal is to identify abnormal memory allocations, illegitimate execution paths, or manipulation of trusted system routines.

## Thread Local Storage (TLS) Callbacks

A key detection mechanism involves the use of **Thread Local Storage (TLS) callbacks**, a feature in the Windows PE (Portable Executable) format that allows developers to specify functions to be executed automatically by the system when threads are created or destroyed, or when the module is loaded or unloaded.

In the context of anti-cheat systems, TLS callbacks are strategically valuable because they are executed **before the application's main entry point is reached**, and even **before**

**asynchronous procedure calls (APCs)** are processed. This early execution allows anti-cheat software to inspect and validate the runtime environment at an extremely privileged stage of process initialization.

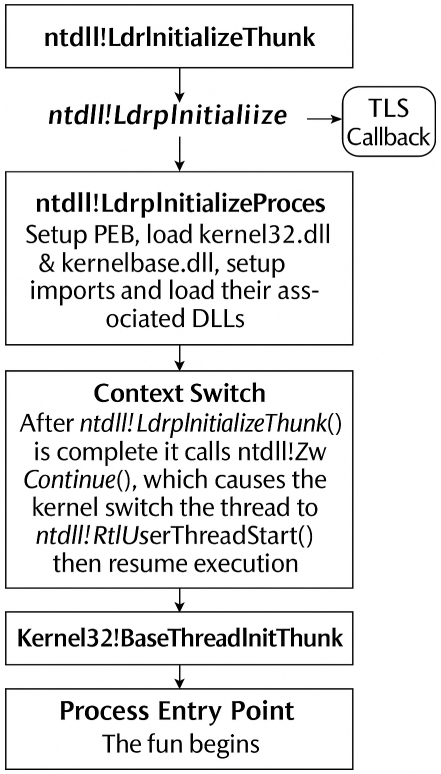


Figure 5.2: Windows thread initialization flow with TLS Callback highlighted.

- On thread creation, the Windows loader invokes TLS callbacks *within* the routine `ntdll!LdrpInitializeProcess`, giving anti-cheat DLLs a head start to perform memory integrity checks or verify thread origins.
- If an injected DLL includes its own TLS callback, it may be triggered at load time—potentially revealing its presence to the anti-cheat if not properly concealed or removed.

Figure 5.2 illustrates the initialization sequence of a thread in Windows. The TLS callback is highlighted to show its execution point **before APC dispatch** and **long before reaching the game's code**. This makes it a powerful mechanism for early detection of injected modules or tampered execution states.

For cheat developers, understanding the timing of TLS execution is critical. If not disabled or manually stripped from injected DLLs, a TLS callback can inadvertently betray the presence of the cheat before it even begins executing its logic. Consequently, many modern cheats using manual mapping techniques explicitly remove the TLS directory from the PE header to avoid this detection vector.

## Start Address Integrity Checks

Another powerful detection primitive involves analyzing the **start address** of threads. Threads that begin execution in anonymous memory or outside known game modules are flagged as suspicious. This method is particularly effective against simple shellcode execution or poorly concealed DLL injections.

## VMT Hook Detection

Valve Anti-Cheat (VAC) is capable of detecting manipulations to **virtual method tables (VMTs)**—a powerful technique commonly used by cheats to hijack the behavior of in-game objects. In object-oriented game engines, many entities expose functionality through virtual methods. By modifying the VMT of such an object, a cheat can effectively redirect calls to its own code, gaining precise control over rendering, physics, and logic routines.

However, VMT hooking leaves detectable traces in memory. Since VMTs are stored in writable memory regions and follow well-defined structural patterns, VAC can perform **pattern recognition** and **integrity verification** to identify tampering. This includes checking whether VMT pointers still reference legitimate memory ranges, or if the functions they point to fall outside of expected modules.

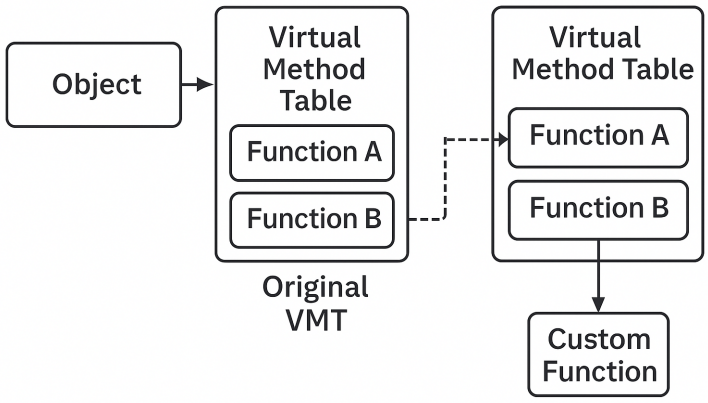


Figure 5.3: Illustration of Virtual Method Table (VMT) Hooking. Function A is redirected to a custom implementation by modifying the object's VMT.

To evade detection, some cheats attempt to clone the original VMT, apply hooks to the clone, and then swap the object's VMT pointer. Others periodically restore and reapply the hook depending on the scan timing. In more recent developments, some developers have abandoned VMT hooking altogether in favor of inline detours or Return-Oriented Programming (ROP) chains, which are harder to attribute to specific objects.

# 4 Evasion Techniques Used by Cheats

To remain undetected, cheat developers employ advanced evasion strategies. These techniques are specifically designed to circumvent TLS callbacks, start address validation, and memory integrity checks.

## Suppressing TLS Callbacks

One method to bypass TLS-based detection is to remove or disable the TLS callback table in the injected DLL. This can be achieved by:

- Manually modifying the PE header to erase TLS structures.
- Avoiding the use of new threads, thus preventing callback invocation.

## Thread Hijacking and Return Address Spoofing

**Thread hijacking** avoids the creation of new threads by taking control of an existing game thread. The cheat suspends a legitimate thread, modifies its instruction pointer, and redirects it to execute the injected payload.

To further mask execution, cheats employ **return address spoofing**, which fakes the origin of a function call. By placing a plausible return address on the stack, the cheat deceives anti-cheats into believing that execution flow originated from a legitimate module.

## Start Address Redirection via ROP Gadgets

One advanced technique used by cheat developers to bypass anti-cheat protections is through the use of **ROP (Return-Oriented Programming) gadgets**. To understand this, think of ROP gadgets as tiny, pre-existing snippets of code—just a few instructions long—that already live in trusted software modules, like the game executable or system libraries. These snippets typically end with an instruction that passes control to another location, such as `ret` (return from function) or `jmp reg` (jump to the address in a register).

Instead of calling malicious code directly—which would be easy for an anti-cheat system to detect—cheats can build a fake function call using these gadgets. This involves creating a small code section called a *call stub* that fakes a legitimate function return. This stub pushes a *spoofed return address* onto the stack (pointing to one of the trusted gadgets) and then jumps to the real payload. From the outside, the resulting execution path looks normal, making it hard to detect through common anti-cheat techniques like return address validation or call stack inspection. This is particularly effective against Valve's anti-cheat engine (VAC), which performs such checks.

**Stack Alignment and Shadow Space.** In 64-bit Windows environments, function calls must follow strict rules. One of these rules involves aligning the stack pointer and reserving 32 bytes of temporary space for the called function (called *shadow space*). The cheat's call stub must respect these rules to avoid crashing or triggering red flags. Here is an example of a minimal x64 stub:

```
sub rsp, 0x28                ; align stack and reserve shadow space
mov rax, 0xFFAAFFAAFFAAFFAA ; spoofed return address (ROP gadget)
push rax
mov rax, 0xFFAAFFAAFFAAFFAA ; actual target function
jmp rax
```

This stub ensures that when the function is done, the processor returns to a safe-looking address (the gadget), not to suspicious or injected code.

**What Makes a Good Gadget?** A **ROP gadget** is simply a small piece of existing code that ends in a control-transfer instruction, usually found inside legitimate modules. Because these are already loaded into memory, using them avoids detection that would arise from injecting new code.

Cheats locate these gadgets by **pattern scanning**—searching memory for specific byte patterns that represent useful instruction sequences. The goal is to find gadgets that:

- Adjust the stack pointer correctly, e.g., to undo the shadow space allocation.
- Avoid modifying important CPU registers used by the function being called.
- Provide a clean and believable return path for the processor to follow.

Here are examples of commonly used gadget patterns and what they do:

```
add rsp, 0x28
ret
```

This gadget fixes the stack after the shadow space and performs a clean return. It's ideal for spoofing legitimate execution flow.

```
pop rcx
add rsp, 0x20
ret
```

This one pops a value into a register (e.g., an argument), adjusts the stack, and returns. It can be useful when additional cleanup or setup is needed.

```
add rsp, 0x20
pop rax
```

```
jmp rax
```

Instead of returning, this gadget jumps directly to a target address stored in `rax`. It serves as a *trampoline*—a way to continue execution without relying on a normal return, which can be risky in some setups.

These gadgets are either hardcoded into the cheat (if their locations are known ahead of time) or dynamically found at runtime based on memory analysis.

**x86 Considerations.** On 32-bit (x86) systems, this technique becomes trickier due to the different calling conventions in use—such as `_cdecl`, `_thiscall`, or `_fastcall`. Each defines how arguments are passed and who is responsible for cleaning up the stack.

Below is a stub tailored for a `_thiscall` function, which is commonly used for C++ member functions on Windows:

```
sub esp, 0x0C
mov eax, 0xFFAAFFAA        ; spoofed return address
mov [esp], eax
mov eax, [esp+0x10]
mov [esp+4], eax
mov eax, [esp+0x14]
mov [esp+8], eax
mov eax, 0xFFAAFFAA        ; actual target function
jmp eax
```

In this example, the return address is placed at the top of the stack, and the arguments are manually positioned to match what the target function expects. This careful setup ensures the cheat integrates smoothly into the normal execution flow, even under the scrutiny of stack-based detection mechanisms.

**Detection Vectors.** Although this technique can bypass static stack validation, it is not impervious to detection. Anti-cheat engines can perform **stack walking**—analyzing the call stack at runtime—to identify anomalies such as unexpected return addresses or invalid module references. To counteract this, some cheats implement post-call cleanup via chained gadgets or trampolines that restore a believable call chain before returning.

**Practical Implementations.** Open-source projects like `x86RetSpoof` by danielkrupinski encapsulate these techniques in reusable tooling for x86-based games. However, bespoke implementations remain common in private cheat circles, especially in paid cheats (*P2C*) for titles like CS:GO and CS2.

# 5 Extended VAC Capabilities and Developer Countermeasures

Beyond memory inspection and thread analysis, VAC includes a broader set of capabilities that significantly increase its detection surface.

## Observed Detection Capabilities

Community research and reverse engineering have revealed that VAC likely performs:

- **File System Scanning**: Searches for known cheat artifacts and patterns.
- **Process and Memory Scanning**: Inspects all running processes for injection, shellcode, and suspicious modules.
- **Registry Inspection**: Identifies unusual startup entries or configuration keys.
- **Handle Enumeration**: Flags abnormal access to game-related objects.
- **Hook and Signature Scanning**: Detects common hook patterns and byte signatures from known cheat loaders.

These mechanisms work in concert with behavioral monitoring and cloud-based analysis, making VAC highly resilient to static evasion alone.

## Common Countermeasures Used by Cheat Developers

In response, professional cheat developers implement an array of countermeasures designed to reduce their attack surface and mislead detection heuristics:

- **String Encryption**: Prevents signature-based scanning of identifiable strings.
- **Window and Module Name Randomization**: Avoids static blacklisting and manual heuristics.
- **Polymorphic Code**: Constantly mutates the cheat's binary structure to evade pattern recognition.
- **Fileless Execution**: Prevents disk-based detection by streaming the cheat entirely into memory.
- **Memory Cleansing**: Removes metadata and headers after injection to eliminate identifiable traces.
- **Registry Hygiene**: Ensures no persistent keys or logs remain after execution.
- **Scan Spoofing**: Hooks or replaces VAC scanning routines with fake responses to conceal activity.

These techniques are essential for achieving long-term undetectability in highly monitored environments such as Valve's matchmaking infrastructure.

DLL injection remains the foundation of modern cheat development in competitive games. Techniques such as Manual Mapping offer powerful stealth capabilities, enabling deep integration with the game runtime. However, anti-cheat systems like VAC employ increasingly sophisticated detection methods, including TLS callbacks, memory integrity scanning, VMT hook detection, and behavioral analysis.

---

# 6. FUNCTION HOOKING AND TRAMPOLINE-BASED INTERCEPTION

## Overview

At the core of our cheat framework lies a fundamental technique: function hooking. This allows us to intercept calls to internal game functions, modify their behavior, and optionally resume execution without breaking the application flow. The most stable and undetectable method used is known as the **trampoline hook**.

Trampoline hooking provides a powerful and safe way to redirect execution to custom payloads while preserving the original function's behavior. In this chapter, we explain what trampoline hooks are, how they work under the hood, and how we implement them using the `MinHook` library.

## 1 What is a Trampoline Hook?

A trampoline hook works by modifying the beginning of a target function to insert a jump to a custom handler (our hook function). Since this overwrites a few bytes of the original function, we first save these instructions into a new piece of memory—the *trampoline*. After our hook executes, we call this trampoline to resume normal function execution.

The full process consists of four main steps:

1. **Original Function (Hooked)** – We patch the first few bytes with a jump to our relay.
2. **Relay Function** – Transfers control to our actual hook payload.
3. **Hook Payload** – Performs cheat logic and calls the trampoline.
4. **Trampoline** – Executes the original instructions that were overwritten, then jumps back to the original function.

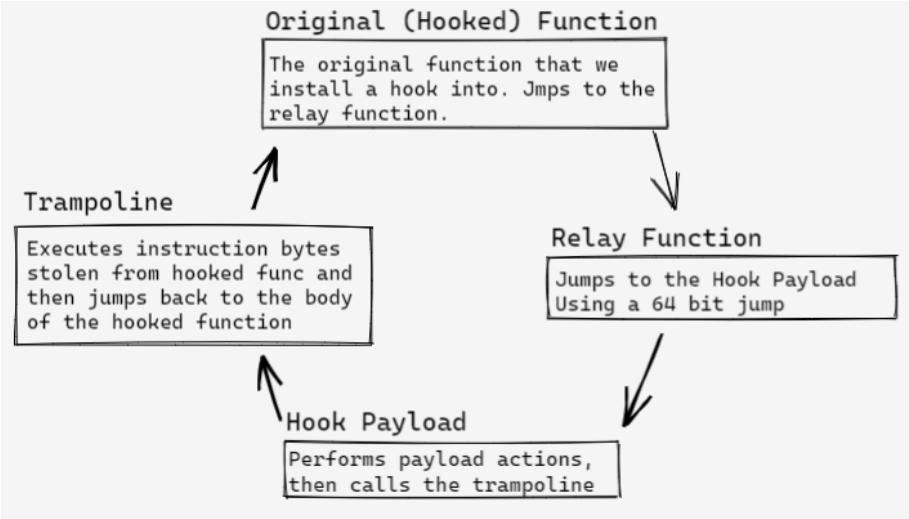


Figure 6.1: Flow of execution in a trampoline hook setup.

## 2 Reverse Engineering and Function Discovery

Before hooking any functions, we must first identify which functions within the game are responsible for the behaviors we want to manipulate. To do this, we reverse engineered several of the game's dynamic link libraries (DLLs). The most useful among them was `client.dll`, as it contains the majority of the code executed on the client-side rather than the server.

Once interesting functions were identified by name and behavior—often using reverse engineering tools such as IDA Pro or Ghidra—we obtained their function signatures and analyzed how they operated.

To ensure our hook could find the same function across different executions (even when Address Space Layout Randomization, or ASLR, shifts their locations), we extracted a unique byte pattern, or **signature**, from each target function. This sequence of bytes acts as a fingerprint that we can use to scan memory for that function at runtime.

The injected DLL uses these patterns to locate the functions dynamically every time the game starts. Once found, our hooking mechanism is deployed.

## 3 Why Trampoline Hooks Are Effective

Trampoline hooks have a major advantage over inline or detour hooks: they preserve the original control flow and return address. This means:

- The original function executes normally after our payload.
- Return address checks performed by anti-cheat systems (like VAC) are bypassed, since the call stack and return address remain consistent with expected behavior.

- We do not inject code into unfamiliar memory regions, reducing detection surface.

# 4 Implementing Hooks with MinHook

To hook functions safely and efficiently, we use **MinHook**, a lightweight and open-source x86/x64 API hooking library for Windows.

Before installing the hook, we first define the structure of the original function inside a dedicated class. This class also contains the trampoline pointer and our custom hook implementation. For example:

```
class WorldModulation {
private:
    static ByteColor ColorCache;

public:
    typedef void* (__fastcall* ModulateWorldColorFn)(
    CAggregateSceneObjectWorld*, void*);
    static ModulateWorldColorFn oModulateWorldColor;
    static void* __fastcall hModulateWorldColor(
    CAggregateSceneObjectWorld* pAggregateSceneObject, void* a2);
};

WorldModulation m_WorldModulation;
```

This setup provides a clean and organized structure where:

- We define the function type via a typedef.
- We store a pointer to the original (trampoline) function.
- We implement our custom hook logic inside a static method.

Then we proceed with the actual installation using MinHook:

```
void* __fastcall WorldModulation::hModulateWorldColor(
    CAggregateSceneObjectWorld* pWorld, void* a2) {
    ByteColor Color = Visual::ToByteColor(Globals::worldModulationColor)
    ;

    if (Globals::worldModulation && Color != ColorCache) {
        for (int i = 0; i < pWorld->count; i++) {
            auto& obj = pWorld->array[i];
            obj.r = Color.r;
            obj.g = Color.g;
            obj.b = Color.b;
```

```
        }
        ColorCache = Color;
    }

    return oModulateWorldColor(pWorld, a2); // Call trampoline
}

void InitHook() {
    MH_Initialize();
    MH_CreateHook((LPVOID)TargetAddress, &WorldModulation::
    hModulateWorldColor, (LPVOID*)&WorldModulation::oModulateWorldColor);

    MH_EnableHook((LPVOID)TargetAddress);
}
```

# 5 Conclusion

Trampoline hooking provides a stable and stealthy way to manipulate game logic without crashing the game or alerting anti-cheat systems. By preserving the return address and execution flow, it forms the backbone of our entire cheat architecture.

The process begins with reverse engineering target DLLs—most notably `client.dll`—to locate and understand the relevant functions. Then, byte-level patterns are extracted to scan and identify those functions reliably at runtime, enabling us to inject hooks safely and consistently. Finally, MinHook allows us to deploy trampoline hooks with minimal footprint, organized in clearly defined classes for maintainability.

In the next chapter, we will explore how to discover and select appropriate functions to hook, particularly those related to rendering, game logic, and player state.

---

# 7. INTERNAL GAME STRUCTURE: LOGIC BEHIND COUNTER-STRIKE 2

## 1 Understanding Entity Lists in Video Games

In modern game engines—particularly those designed for real-time, multiplayer environments—an **entity list** is a fundamental data structure responsible for managing all interactive objects within the game world. These entities include players, non-player characters (NPCs), projectiles, grenades, environmental props, and more.

At the core, each entity is an instance of a class containing essential data such as position, health, team ID, state flags, and animation or input components. The engine maintains and updates these entities each frame to simulate gameplay logic, render visuals, process collisions, and evaluate state-based behavior such as death, visibility, or interaction.

### 1.1 Common Structures for Entity Lists

Entity lists are implemented using various structures, depending on the performance needs and memory model of the engine:

- `Array of Entity Objects` – A flat, contiguous layout offering fast access, but limited flexibility.
- `Array of Pointers` – A static array referencing dynamically allocated entities. Used frequently in older engine architectures.
- `std::vector` – A dynamic container suitable for runtime entity creation and destruction.
- `Linked List` – Each node points to the next entity, providing memory flexibility at the cost of linear traversal time.
- `HashMap / Tree` – Enables fast lookup and categorization by entity ID or class type.

While early iterations of the Source engine (e.g., CS:GO) utilized a flat pointer array, the modern Source 2 engine in Counter-Strike 2 introduces a **paged entity system**. This structure maps entity handles to memory pages, offering greater scalability and safety across dynamically allocated objects.

# 2 Entity Organization in Counter-Strike 2

In *Counter-Strike 2*, every in-game object—be it a player, weapon, projectile, or environmental element—is represented as a dynamic **entity**. Internally, these entities are managed through unique *handles*, compact structures that encode both the entity index and a generation counter. This system enables safe referencing, preventing stale or invalid pointers when entities are destroyed and recreated during gameplay.

To resolve a handle into a valid pointer, the Source 2 engine employs a two-level paging system—conceptually similar to the virtual memory mechanisms used in modern operating systems—allowing efficient access to entity memory while maintaining safety and scalability.

When focusing on player-related data, two core classes emerge as central components:

- **CBasePlayerController** — Contains high-level player metadata such as the sanitized name, Steam ID, input state, and a handle to the corresponding pawn.
- **C_CSPlayerPawn** — Represents the physical player entity within the game world. It includes key gameplay attributes such as position, health, team ID, view angles, and animation state.

Both classes derive from the common superclass **C_BaseEntity**, which serves as the foundation for all entities in the engine. It exposes core functionality and network-relevant properties. Below are simplified, annotated representations of these structures:

### 1. Base Entity – `C_BaseEntity`

```
/**********************************************
 * C_BaseEntity:
 * Base class for all entities in the Source 2 engine.
 * Provides core fields used by all entity types.
 **********************************************/
class C_BaseEntity {
public:
    CHandle<C_BaseEntity>     m_hEffectEntity;      // * Effect entity (e.g. glow, particle)
    CHandle<C_BaseEntity>     m_hOwnerEntity;       // * Owner entity (e.g. weapon holder)
    CHandle<C_BaseEntity>     m_hGroundEntity;      // * Entity this one is standing on (for physics)
    CBodyComponent::Storage_t m_CBodyComponent;     // * Physics and collision representation

    // * ... Additional base class members used engine-wide
};
```

Listing 7.1: C_BaseEntity – Core Class for All Entities

### 2. Player Controller – `CBasePlayerController`

```
/*************************************************************
 * CBasePlayerController:
 * Handles metadata, Steam ID, input state, and pawn reference.
 * Represents the logical (non-physical) part of the player.
 *************************************************************/
class CBasePlayerController : public C_BaseEntity {
public:
    std::string              m_sSanitizedPlayerName;  // * Cleaned player name (UI, logs)
    uint64_t                 m_steamID;               // * 64-bit unique Steam ID
    InputState_t             m_inputState;            // * Current input (WASD, mouse, etc.)
    CHandle<C_CSPlayerPawn>  m_hPawn;                 // * Handle to the in-game player pawn

    // * ... Additional metadata like scores, latency, etc.
};
```

Listing 7.2: CBasePlayerController – Player Metadata and Input Handler

### 3. Player Pawn – `C_CSPlayerPawn`

```
/*****************************************************************
 * C_CSPlayerPawn:
 * Represents the player's in-world presence (rendered and simulated).
 * Stores spatial, animation, and gameplay-critical state.
 *****************************************************************/
class C_CSPlayerPawn : public C_BaseEntity {
public:
    Vector3            m_vPosition;       // * Player's world-space position
    uint32_t           m_iHealth;         // * Health (0 = dead)
    uint32_t           m_iTeamNum;        // * Team ID (e.g., 2 = T, 3 = CT)
    QAngle             m_angViewAngles;   // * Viewing direction (pitch/yaw/roll)
    AnimationState_t   m_AnimState;       // * Animation and pose state
```

```
13
14      // * ... Additional attributes like velocity, inventory, flags, etc.
15  };
```

Listing 7.3: C_CSPlayerPawn – In-World Representation of the Player

This class hierarchy enables a clean separation of concerns between the logical identity and control of the player (controller) and their physical simulation within the world (pawn). This design also accommodates edge cases such as spectator modes, bot logic, and replay systems, where a controller may exist without an active pawn.

Moreover, the architecture reflects a variant of the Entity-Component-System (ECS) paradigm, promoting modularity and flexibility. This pattern is especially beneficial for external tools, including mods, analytics systems, and cheat frameworks.

## 2.1 Structuring Player Data in Practice: The `Players` Class

To organize the resolved player entities in a consistent and performant way, we define a high-level wrapper class called `Players`. This class aggregates all relevant information associated with a player in a single structure:

- The player's in-game entity (`Pawn`), responsible for physical representation and simulation.
- The logical controller (`Controller`), which manages metadata, input, and team identity.
- The observed pawn (`ObserverPawn`), used for deathcam, spectator, or overwatch viewing.

This encapsulation allows us to treat each player as a self-contained unit, streamlining access across features such as ESP overlays, aimbots, spectator tools, or replay systems.

### Entity Aggregation Class – `Players`

```
1   class Players
2   {
3   public:
4       C_PlayerPawn*       Pawn;
5       C_PlayerController* Controller;
6       C_PlayerPawn*       ObserverPawn;
7
8       Players()
9           : Pawn(nullptr), Controller(nullptr), ObserverPawn(nullptr) {}
10
```

```
11      Players(C_PlayerPawn* pawn, C_PlayerController* controller,
        C_PlayerPawn* observerPawn)
12          : Pawn(pawn), Controller(controller), ObserverPawn(observerPawn)
         {}
13  };
```

Listing 7.4: Players – Unified Player Representation

# 3 Entity List Traversal

Having established the structural foundations of player entities, the next step involves enumerating these entities during runtime. This process—commonly referred to as **entity list traversal**—is fundamental for real-time game modification and data extraction.

## 3.1 Entity Handle Relationships

The linkage between `CBasePlayerController` and `C_CSPlayerPawn` is facilitated by the use of compact entity handles—typically implemented as `uint32_t` integers.

- `CBasePlayerController::m_hPawn` points to the controlled pawn.
- `C_CSPlayerPawn::m_Controller` points back to the controlling entity.

This bidirectional association enables fast, stable resolution of the player structure across different modules and states.

## 3.2 Locating and Identifying the Entity List in Memory

In the absence of debugging symbols or SDK access—common in protected or closed-source games—discovering the location and structure of the entity list becomes a crucial part of the reverse engineering process. This list is essential for iterating over active in-game entities such as players, bots, or interactive objects, and retrieving key data like position, health, and team.

The process is typically performed using memory inspection tools such as Cheat Engine, and can later be optimized or automated once the structure is confirmed. A generalized and practical methodology for discovering the entity list dynamically is outlined below.

### Dynamic Discovery Workflow

1. **Scan for the Local Player's Position:** Begin by searching for changing float values corresponding to the local player's coordinates (e.g., `X`, `Y`, `Z`) as you move within the game.

2. **Filter Non-Dynamic or Cached Results:** Disregard addresses that stop updating when the game is paused or the window loses focus. These are often visual proxies or cached rendering data.
3. **Analyze Accessing Instructions:** Use Cheat Engine's "Find out what accesses this address" feature to locate game instructions interacting with the position. Valid update loops will continue accessing these addresses even when the game is idle.
4. **Detect Multi-Entity Access Patterns:** Verify whether the same instruction reads positions of other players. If so, it likely belongs to a loop iterating through a centralized entity container.
5. **Derive Entity Base Addresses:** Subtract the known position offset (e.g., `m_vPosition`) from each accessed address to recover the base pointer to each entity structure.
6. **Identify the Entity List Container:** Search for contiguous memory blocks where these base addresses are stored. Look for uniform spacing between them (e.g., every 0x78 or 0x80 bytes), which often indicates a static array or a paged entity structure.
7. **Validate and Generalize:** Once confirmed, build a loop to iterate over the list using pointer arithmetic and known offsets. From here, you can begin resolving additional properties like health, team, and state flags.

### Entity List Variants: Visual vs. Gameplay-Complete Structures

During memory inspection, it is common to encounter multiple memory regions that resemble entity lists. These structures may vary significantly in terms of the data they expose:

- **Render-Only Lists:** Often contain just positional or interpolation data used by the renderer. These lists update frequently but lack gameplay logic.
- **Gameplay-Relevant Lists:** Contain complete entity structures with health, team, ammo, armor, and state flags—used by core game logic for decision-making and rule enforcement.

While render-only lists can be identified relatively quickly—typically within minutes—the discovery of fully populated gameplay-relevant structures often requires extended investigation. The process may involve correlating memory offsets, validating runtime behavior, or leveraging function-level reverse engineering in disassembly tools.

Advanced reverse engineers often turn to static analysis using tools such as **IDA Pro**, **Ghidra**, or **Binary Ninja**. These enable the inspection of vtables, RTTI metadata, or instruction-level access patterns that point to the actual entity list access logic inside the game binary.

This multi-path approach is extensively documented and discussed by reverse engineering communities. A particularly useful case study can be found in a dedicated thread on the UnknownCheats forum[1], where users share strategies for locating and validating entity lists across multiple game engines.

In conclusion, identifying the correct entity list—whether through dynamic analysis or disassembly-guided pattern matching—is a foundational skill for external tooling. It unlocks the ability to systematically access and manipulate in-game objects with high confidence and reliability, forming the basis for any higher-level system such as ESP, aimbots, or replay parsers.

## 3.3 Reverse Engineering the Engine's Entity Resolution Logic

Through static analysis of the CS2 binary, specifically within the `client.dll`, one can uncover the internal logic used to resolve handles. The following C pseudocode, extracted from Ghidra, corresponds to the function responsible for locating entities within the paged entity list:

```
Entity_t *EntityList(longlong EntityListPointer, uint EntityListIndex)
{
    Entity_t *Entity;
    longlong EntitySublist;

    if ((((EntityListIndex < 32767) && (EntityListIndex >> 9 < 64)) &&
         (EntitySublist = *(EntityListPointer + 16 + (EntityListIndex >>
     9) * 8), EntitySublist != 0))
        && ((Entity = EntitySublist + (EntityListIndex & 511) * 120,
    Entity != NULL &&
             ((*(Entity + 2) & 32767) == EntityListIndex))))
    {
        return *Entity;
    }
    return 0;
}
```

Listing 7.5: Accessing an Entity in the Entity List

The algorithm follows these main steps:

1. **Index Validation**: First, the index `EntityListIndex` is validated to ensure it falls within a valid range. Specifically:

[1] https://www.unknowncheats.me/forum/general-programming-and-reversing/564058-entity-list.html

- `EntityListIndex < 32767` ensures the index does not exceed a predefined maximum value. The game defines that 32766 is the maximum number of entities it can handle at any given time.
- `EntityListIndex >> 9 < 64` uses a right shift operation, which effectively divides the index by 512, ensuring that the index belongs to one of the 64 available sublists. In this way, each list can contain 512 entities.

2. **Accessing the Sublist**: The function calculates the appropriate sublist that contains the entity:
   - `EntitySublist = *(EntityListPointer + 16 + (EntityListIndex >> 9) * 8)` computes the address of the specific sublist by using the shifted index to determine its location in memory.
   - The condition `EntitySublist != 0` ensures the sublist is valid (i.e., it exists and can be accessed).
3. **Locating the Entity Within the Sublist**: If the sublist is valid, the algorithm calculates the position of the entity within that sublist:
   - `Entity = EntitySublist + (EntityListIndex & 511) * 120` calculates the position of the entity inside the sublist. The bitwise AND operation `EntityListIndex & 511` extracts the last 9 bits of the index, which helps pinpoint the entity's exact position within the sublist.
   - The condition `Entity != NULL` ensures that the entity actually exists, and `(*(Entity + 2) & 32767) == EntityListIndex` checks that the entity's index matches the requested one.
4. **Returning the Entity**: Once all conditions are met, the algorithm returns the entity. If any of the conditions fail, the function returns `0`, indicating that the entity could not be found.

### 3.3.1 Alternative Method: Hooking `OnAddEntity` and `OnRemoveEntity`

While iterating over the full entity list is a viable approach—as demonstrated in the previous section—it has limitations, especially in terms of safety and performance. A naive traversal risks accessing null or invalid entities, particularly in scenarios where:

- A player has recently disconnected.
- An entity was destroyed or reassigned (e.g., team switch or respawn).
- The entity memory is not yet initialized by the engine's internal loop.

To mitigate these risks, we employ a more reliable and event-driven technique: **hooking the engine's entity lifecycle callbacks**. Specifically, we intercept two key functions—

`OnAddEntity` and `OnRemoveEntity`—which the engine internally invokes whenever an entity is added to or removed from the active game world.

By intercepting these functions, we maintain a synchronized internal representation of the entity list, without having to traverse memory manually every frame.

### Hooked Callbacks for Entity Lifecycle

```
void HooksManager::OnAddEntity::hOnAddEntity(__int64 CGameEntitySystem,
    void* entityPointer, int entityHandle)
{
    oOnAddEntity(CGameEntitySystem, entityPointer, entityHandle);

    std::string schemaName = iGameEntitySystem->GetSchemaName(
    entityPointer);

    if (schemaName == "C_CSPlayerPawnBase")
    {
        iGameEntitySystem->PawnMap[entityHandle] = (C_PlayerPawn*)
    entityPointer;
    }

    if (schemaName == "c_cs_observer_for_precache")
    {
        iGameEntitySystem->ObserverMap[entityHandle] = (C_PlayerPawn*)
    entityPointer;
    }

    if (schemaName == "CBasePlayerController")
    {
        iGameEntitySystem->ControllerMap[entityHandle] = (
    C_PlayerController*)entityPointer;
    }
}
```

Listing 7.6: Hook: OnAddEntity

### Lifecycle-Driven Synchronization

With this approach, the cheat framework reacts to entity creation and destruction in real time, mirroring the game's own internal management. This results in several key benefits:

- **Performance:** No need to scan memory every frame.
- **Accuracy:** No risk of accessing invalid or outdated entity pointers.

- **Extensibility:** Enables features like lifecycle analytics or event logging.

### Initial Synchronization Step

It is worth noting that this hook-based method alone does not populate the entity list retroactively. Therefore, upon joining a game that has already started, we must perform a one-time full traversal of the entity list to capture entities that were added prior to our hooks being installed.

Once this initial population step is completed, entity state is fully managed via the `OnAddEntity` and `OnRemoveEntity` hooks—ensuring consistency, minimal overhead, and maximal safety.

---

# 8. CHEAT FEATURES IMPLEMENTED IN OUR CHEAT FOR CS2

This chapter provides a comprehensive analysis of the **cheat features** we have successfully implemented within our *cheat*. We will explore their objectives, the underlying logic governing their operation, and the technical implementation that enables them to function correctly. Each section will present **code snippets** that illustrate key parts of our implementation, allowing for a deeper understanding of how we interfaced with the CS2 engine. Furthermore, we will outline the specific **functions we have hooked**, demonstrating how we gain control over critical game processes to manipulate rendering, aiming, and other essential aspects of gameplay.

| Functionality | Description |
|---|---|
| **Chams** | Modifies the rendering pipeline to apply custom materials to in-game models, enhancing visibility. |
| **ESP (Extra Sensory Perception)** | Extracts and displays real-time enemy information, providing a tactical advantage. |
| **Aimbot** | Automates aiming mechanisms to improve accuracy and precision. |
| **Anti-Aim** | Manipulates player model orientation to mislead enemy aimbots and disrupt targeting. |

Table 8.1: Core Functionalities Overview

There exist multiple approaches to implementing each of these cheat features, ranging from simple external overlays to deep internal hooks that modify game memory and execution flow. Given the time constraints of this work and the limited publicly available documentation on cheat development, we have opted for an approach that provides the best balance between effectiveness and stealth.

The lack of official documentation and the inherently secretive nature of cheat development meant that much of our implementation had to be based on reverse engineering and extensive experimentation. Instead of relying on external overlays, which are more susceptible to detection and performance limitations, or resorting to aggressive memory manipulation, which significantly increases the risk of being flagged by anti-cheat systems, we chose to integrate directly with the game's internal systems.

We begin by exploring **Chams**, one of the most visually impactful modifications. This technique alters the appearance of in-game models by replacing their default materials with custom ones, effectively enhancing player visibility. Unlike traditional *ESP* methods, which rely on external overlays that merely draw information on the player's screen, Chams modify the rendering process within the engine itself. This allows for an entirely integrated solution that operates without any **performance impact**, making it one of the most effective and undetectable visual enhancements available.

In the following sections, we will dissect the logic behind our Chams implementation, detailing how we intercept key rendering functions, modify material properties, and dynamically apply custom textures to selected in-game entities. Each aspect of the implementation will be supported by relevant code snippets, providing an in-depth look at how we have programmed this feature.

# 1 Chams: Enhancing Visibility Through Engine-Based Rendering

Chams (*chameleon skins*) is a rendering manipulation technique that enables the replacement of standard in-game materials with custom-designed ones, significantly enhancing model visibility and allowing players to track enemy movements with greater ease regardless of lighting conditions, distance, or obstacles that would typically obscure their position.

Unlike external overlays, which function independently from the game and merely display additional information on the screen, Chams operate directly within the game engine, modifying the rendering logic at its core to ensure seamless integration and optimal performance without introducing additional processing overhead.

By leveraging hooking techniques within CS2's material system, it becomes possible to dynamically inject new textures and visual effects that are not present in the base game, allowing for a level of customization that surpasses conventional graphical modifications while maintaining the efficiency and fluidity of the rendering pipeline.

To achieve this level of modification, we implemented a multi-step process that ensures full integration with the game's rendering system. This process consists of the following key components:

- **Extracting Default Materials from the Game Engine** – Understanding and retrieving the default materials used by the game.
- **Generating Custom Materials** – Creating and defining new textures and visual effects that will replace the existing materials.

- **Hooking CreateMaterial and Injecting Custom Materials** – Intercepting the game's material creation functions to replace default materials with our customized versions dynamically.
- **Hooking OnDrawModelExecute and Overriding Materials** – Modifying the rendering function responsible for drawing models to enforce the application of custom materials in real-time.

In the following sections, we will provide a detailed explanation of each of these steps, outlining how they contribute to the seamless integration of Chams into the CS2 rendering pipeline. We will delve into the technical specifics of how default materials are extracted, how custom materials are generated, and how we hook into the rendering functions to ensure that these modifications persist across all game instances.

## 1.1 Extracting Default Materials from the Game Engine

To fully understand how materials are structured and applied in CS2, we conducted extensive **debugging of the rendering engine**. By hooking into the **OnDraw** function, we intercepted the material creation process and successfully extracted the data of a default in-game material.

The following is an example of a material retrieved directly from the CS2 engine:

```
<!-- kv3 encoding:text:version{e21c7f3c-8a33-41c5-9977-a76d3a32aa0d}
    format:generic:version{7412167c-06e9-4698-aff2-e63eb59037e7} -->
    {
        shader = "csgo_effects.vfx"
        g_tColor = resource:"materials/dev/
    primary_white_color_tga_21186c76.vtex"
        g_tNormal = resource:"materials/default/
    default_normal_tga_7652cb.vtex"
        g_flColorBoost = 30
        g_flOpacityScale = 245.55
        g_vColorTint = [7.0, 7.0, 7.0, 0.37522]
    }
```

Listing 8.1: Extracted Default Material from CS2

The extracted material follows the KV3 format, a modernized data structure introduced in Source 2 to replace the older KeyValues system used in Source 1.

KV3 allows for efficient serialization of structured data, supporting hierarchical organization and multiple data types, making it ideal for defining game assets such as materials, models, and particle effects. The Source 2 engine interprets these material definitions at runtime, applying the specified shader, texture maps, and rendering properties dynamically. By understanding this format, we gain full control over how materials are created

and manipulated in CS2, enabling us to inject entirely new visual elements into the game without modifying pre-existing assets.

Further exploration of this format, along with an in-depth analysis of its structure and applications, was possible by studying the official Source 2 Developer Wiki, which provides extensive documentation on how Source 2 processes materials, shaders, and other graphical assets. This resource offered crucial insights into how KV3 interacts with the rendering engine, particularly in the way materials are compiled and utilized in real time.

By modifying this format, we successfully generated a variety of new materials, each possessing unique properties suited for different visual effects, allowing us to experiment with different rendering techniques and further refine our implementation of Chams in CS2.

## 1.2 Generating Custom Materials

After extracting and analyzing a default material from the game, we proceeded to experiment with different configurations to generate a collection of materials optimized for various Chams effects.

The goal of this process was to develop materials capable of altering in-game visibility while maintaining the integrity of CS2's rendering engine. By modifying specific attributes such as `g_flFresnelExponent` for light reflection, `g_flOpacityScale` for transparency, and the blending mode flags, we achieved a variety of effects including glow effects, metallic reflections, and enhanced visibility through obstacles.

```
std::vector <const char *> MaterialInfo = {
    szVMatBufferWhiteVisible,    szVMatBufferWhiteInvisible,
    szVMatBufferIlluminateVisible,    szVMatBufferIlluminateInvisible,
    szVMatBufferIlatex,        szVMatBufferIlatexInvisible,
    szVMatBufferMetalic,        szVMatBufferMetalicInvisible,
    szVMatBufferGlowVisible,    szVMatBufferGlowInvisible
};
```

Listing 8.2: List of Custom Materials

Each of these materials was assigned a name that reflects its primary visual characteristic to facilitate its integration into our cheat system. By storing these materials in a **hashmap**, we enable efficient retrieval and reuse whenever new materials need to be applied.

Since materials in CS2 are defined as structured strings that determine their rendering properties, we ensure that our injected materials adhere to this format. This allows us to seamlessly integrate custom materials into the game's rendering pipeline by hooking into the `CreateMaterial` function and dynamically replacing standard materials with our own.

One of the most visually distinctive materials we developed was the **Glow Illuminated** effect. This material was designed to create a glowing outline around player models by leveraging CS2's built-in material masks and fine-tuning fresnel-based lighting parameters. The final definition of this material is shown below:

```
static static constexpr char szVMatBufferGlow2Visible[] =
R(<!-- kv3 encoding:text:version{e21c7f3c-8a33-41c5-9977-a76d3a32aa0d}
    format:generic:version{7412167c-06e9-4698-aff2-e63eb59037e7} -->
    {
        shader = "csgo_effects.vfx"
        g_tColor = resource:"materials/dev/
    primary_white_color_tga_21186c76.vtex"
        g_tNormal = resource:"materials/default/
    default_normal_tga_7652cb.vtex"
        g_tMask1 = resource:"materials/default/default_mask_tga_344101f8.
    vtex"
        g_tMask2 = resource:"materials/default/default_mask_tga_344101f8.
    vtex"
        g_tMask3 = resource:"materials/default/default_mask_tga_344101f8.
    vtex"
        g_flOpacityScale = 0.45
        g_flFresnelExponent = 0.75
        g_flFresnelFalloff = 1
        g_flFresnelMax = 0.0
        g_flFresnelMin = 1
        F_ADDITIVE_BLEND = 1
        F_BLEND_MODE = 1
        F_TRANSLUCENT = 1
        F_IGNOREZ = 0
        F_DISABLE_Z_WRITE = 0
        F_DISABLE_Z_BUFFERING = 0
        F_RENDER_BACKFACES = 1
        g_vColorTint = [1.0, 1.0, 1.0, 0.0]
});
```

Listing 8.3: Glow Illuminated Material Definition

This configuration takes advantage of Source 2's built-in rendering properties to produce a highly effective glow effect. The fresnel parameters control how light interacts with the surface, ensuring that the glow intensity changes dynamically based on the player's viewing angle. Additionally, the use of multiple texture masks helps the glow blend naturally into the surrounding environment, making it appear as a seamless part of the game's visual style rather than an artificial overlay.

By refining these materials through iterative testing and real-time observation, we successfully developed a collection of custom visual effects that seamlessly integrate into CS2's rendering engine. This process not only allowed us to enhance player visibility but also provided deeper insights into the flexibility of Source 2's material system. The ability to dynamically inject and apply new materials in real time offers significant advantages, as it enables rapid adjustments to the cheat's visual effects without requiring modifications to game files or precompiled assets.

Through the careful selection of material properties and the precise manipulation of CS2's rendering functions, we have created a system that allows for highly customizable visual enhancements. These materials serve as the foundation for our Chams implementation, which will be further detailed in the following sections.

## 1.3 Hooking CreateMaterial and Injecting Custom Materials

To dynamically create and register materials in CS2, we hooked into the **CreateMaterialFunction**, allowing us to manipulate how the game initializes and stores materials.

```
static int64_t (__fastcall *CreateMaterialFunction)(void *, void *,
    const char *, void *, unsigned int, unsigned int);
```

Listing 8.4: Hooking CreateMaterial Function

With this function, we dynamically generated and injected new materials into the game:

```
CMaterial2 *CreateCustomMaterial(const char *materialName, const char *
    materialData) {
    CKeyValues3 *pKeyValues = CreateMaterialResource();
    if (!pKeyValues) return nullptr;

    LoadKeyValues(pKeyValues, nullptr, materialData, nullptr, nullptr,
    nullptr, nullptr, nullptr, "");
    CMaterial2 *customMaterial = nullptr;
    CreateMaterialFunction(nullptr, &customMaterial, materialName,
    pKeyValues, 0, 1);
    return customMaterial;
}
```

Listing 8.5: Creating a Custom Material in Memory

## 1.4 Loading KV3 Materials into the Game

After defining the custom materials, they must be properly loaded into the game engine so that they can be recognized and applied during rendering. This process involves parsing the KV3 data and ensuring that the Source 2 engine interprets it correctly. Without

this step, the materials would remain as raw data in memory and would not be accessible for rendering operations. The following function is responsible for handling this process:

```
bool Visual::Chams::LoadKV3 (CUtilsBuff *buff, CKeyValues3 *CkeyVal)
{
    KV3ID_t kv3ID = KV3ID_t ("generic", 0x41B818518343427E, 0
    xB5F447C23C0CDF8C);
    return LoadKeyValues (CkeyVal, nullptr, buff, &kv3ID, nullptr,
    nullptr, nullptr, nullptr, "");
}
```

Listing 8.6: Loading a KV3 Material into the Game

This function plays a critical role in ensuring that the Source 2 engine correctly processes and integrates the KV3 material data. The most notable aspect of this function is the initialization of the `KV3ID_t` object, which serves as a unique identifier for the KV3 data format. The string `"generic"` suggests that the format follows a standard KV3 structure. However, the most crucial part of this initialization lies in the two hexadecimal values: `0x41B818518343427E` and `0xB5F447C23C0CDF8C`.

These identifiers are likely used by the Source 2 engine as a reference signature for processing specific types of KV3 data. The first hexadecimal value, `0x41B818518343427E`, may act as a format validation key that the engine checks to ensure that the data being loaded conforms to the expected KV3 structure. The second identifier, `0xB5F447C23C0CDF8C`, could correspond to a particular category of KV3 files, distinguishing different types of resource files such as materials, models, or particle effects. This mechanism prevents malformed or incompatible KV3 data from being processed, ensuring that only correctly structured material definitions are registered in the engine.

Once the KV3 identifier is established, the function calls `LoadKeyValues`, which processes the KV3 data stored in memory. This function takes the parsed key-value pairs, ensures that the format aligns with the engine's expectations, and then integrates the material into the game's resource management system. The function parameters include a reference to the parsed data, a pointer to the KV3 identifier, and several `nullptr` arguments, which indicate that no additional optional parameters are required in this context.

By executing this function, the Source 2 engine successfully processes and registers the material, making it available for rendering in real-time. The use of predefined hexadecimal identifiers ensures that the KV3 data is correctly formatted and compatible with the rendering system. This approach enables the dynamic application of custom materials, allowing for real-time modifications without requiring precompiled assets or modifications to the game's existing files. The ability to inject materials dynamically provides a powerful method for implementing features such as Chams, enhanced weapon skins,

and other visual modifications while maintaining full compatibility with the game engine.

## 1.5 Hooking `OnDrawModelExecute` and Overriding Materials

To dynamically apply Chams, we intercepted `OnDrawModelExecute`, the function responsible for rendering in-game models. By hooking into this function, we gained control over the rendering process, allowing us to selectively modify materials applied to specific in-game entities. This method ensures that our custom materials are seamlessly integrated into the game's rendering pipeline without introducing additional rendering overhead or external overlays.

### 1.5.1 Filtering Target Entities

Since `OnDrawModelExecute` processes all rendered models, it is necessary to implement a filtering mechanism to ensure that only relevant entities—such as enemy players and weapons—are affected by our Chams system. Without this step, the modification would be applied indiscriminately to all objects in the game, including environmental elements and friendly players, which is neither efficient nor desirable.

To achieve this, we retrieve the schema name of each entity and compare it against known identifiers for player models and weapons. For player models, we check whether the entity corresponds to a `C_CSPlayerPawnBase` instance, which represents in-game characters.

```
auto schemaName = iGameEntitySystem->GetSchemaName(pEntity);
if (strcmp(schemaName.c_str(), "C_CSPlayerPawnBase") == 0) {
    applyCustomMaterial(pMesh);
}
```

Listing 8.7: Identifying Player Models

Similarly, for weapons, we filter out entities corresponding to `C_PredictedViewModel`, ensuring that Chams are also applied to the player's held weapon model.

```
if (strcmp(schemaName.c_str(), "C_PredictedViewModel") == 0) {
    applyCustomMaterial(pMesh, weaponMaterial);
}
```

Listing 8.8: Applying Chams to Weapons

By implementing this filtering mechanism, we ensure that Chams are applied exclusively to relevant gameplay elements, enhancing player visibility without interfering with other rendered objects.

### 1.5.2 Overriding Material Pointers

Once the relevant entities are identified, the next step is to override their assigned materials with our custom Chams textures. This is achieved by modifying the material pointer associated with the target model.

```
arrMeshDraw->CMaterial = *(CMaterial2 **)CustomMaterialMap.at(
    MaterialName);
```

Listing 8.9: Overriding Model Materials

This modification dynamically replaces the original material with one of our preloaded Chams materials stored within the `CustomMaterialMap`. By performing this override at runtime, we ensure that our changes take effect immediately, without requiring any modification to the game's original assets. This approach enables real-time visual adjustments and allows for the implementation of different Chams effects based on user preferences or situational requirements.

By leveraging this hooking technique, we achieve a stealthy and efficient modification of the rendering pipeline, integrating custom materials in a way that is indistinguishable from the game's native rendering behavior. This method not only enhances the effectiveness of Chams but also serves as a foundation for further graphical modifications within the cheat system.

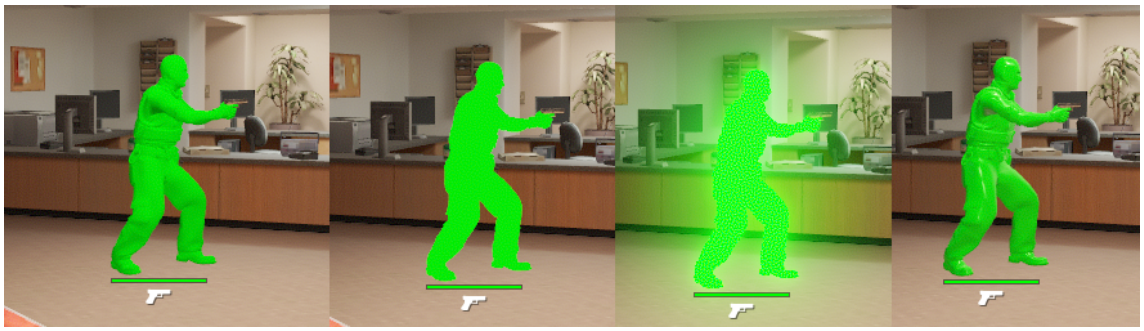

Figure 8.1: Created Chams Materials

### 1.5.3 Visible vs Non-Visible Chams Logic

Once this logic is in place, we can take it a step further and assign different materials to the visible and non-visible parts of the enemy player model. This allows us to render, for example, the visible part of the player in one color (e.g., purple) and the non-visible part (e.g., behind walls) in another (e.g., yellow).

Since this rendering is only visible locally to the cheat user and not to other players in the match, it enables us to appear as a legitimate player with significantly better awareness and reflexes. By clearly seeing which parts of an enemy are exposed and which are not, we can ensure to only target the visible areas.


Figure 8.2: Enter Caption

This reduces the risk of suspicious-looking shots (such as pre-firing through walls or headshots through solid cover) and helps us maintain a ”legit” playstyle that blends in with skilled human behavior. In turn, this greatly decreases the chances of triggering manual bans during demo reviews or overwatch-like systems, where human moderators could flag unnatural behavior.

# 2 Aimbot: Automated Aim to Enemy Players

An aimbot is one of the most fundamental and widely recognized cheat functionalities in competitive first-person shooters. It automates the aiming process by computing the precise trajectory required to target an enemy, thereby bypassing human limitations in reaction time and accuracy. Developing an aimbot for *Counter-Strike 2* (CS2) necessitates a comprehensive understanding of the game's **entity system**, **coordinate transformations**, and **trigonometric principles** to ensure precise target alignment.

However, before implementing the aiming mechanism, it was crucial to first analyze how CS2 internally manages game entities. This understanding was essential since retrieving a player's position and calculating the required angles for the aimbot depend on accessing valid entity data. Without proper entity management, attempting to access a non-existent player could lead to dereferencing null pointers, resulting in critical errors or crashes.

## 2.1 Entity Management: Hooking the Game's Entity System

CS2 maintains an internal **entity system** that is responsible for handling all game objects, including players. Each entity is uniquely identified and assigned a memory address, allowing the game to track its state efficiently. Unlike other dynamic game objects that are continuously updated, player entities in CS2 follow a more static lifecycle: they are loaded into memory once a player connects to a server and remain persistent unless a player disconnects or a new player joins the match. This means that, unlike projectiles or temporary objects, player entities are only modified under specific conditions, reducing unnecessary updates and improving performance.

To reliably access player data and maintain an up-to-date reference to all active players, we identified and hooked into two critical functions within the game's entity management system:

- **OnAddEntity** – Responsible for adding new entities to the game. When a player joins a match, the game dynamically allocates memory for a new entity, assigns it a unique identifier, and registers it in the entity system.
- **OnRemoveEntity** – Handles the cleanup and removal of entities when they leave the match. When a player disconnects or is kicked, the entity system marks their associated entity for deletion and releases the allocated memory.

By intercepting these functions, we effectively track player entities and store their references in a **hashmap**. This hashmap allows us to maintain real-time access to entity data while preventing invalid memory access. Since player entities are only modified when a new player joins or an existing player disconnects, the hashmap remains stable and does not require frequent updates, unlike entity lists that need to be re-queried every frame.

We implemented hooks for `OnAddEntity` and `OnRemoveEntity` to ensure our entity tracking remains consistent throughout a match. By leveraging **hashmaps**, we can efficiently store and retrieve player-related entities, reducing overhead and ensuring a seamless interaction with the game's entity system. Furthermore, this approach minimizes reliance on manually finding offsets, as entity references remain valid until explicitly removed by the engine.

```
void HooksManager::OnAddEntity::hOnAddEntity(__int64 CGameEntitySystem,
    void *entityPointer, int entityHandle) {
    oOnAddEntity(CGameEntitySystem, entityPointer, entityHandle);

    if (strcmp(iGameEntitySystem->GetSchemaName(entityPointer).c_str(),
    "C_CSPlayerPawnBase") == 0) {
        iGameEntitySystem->PawnMap.insert(std::make_pair(entityHandle, (
    C_PlayerPawn *)entityPointer));
    }
    if (strcmp(iGameEntitySystem->GetSchemaName(entityPointer).c_str(),
    "CBasePlayerController") == 0) {
        iGameEntitySystem->ControllerMap.insert(std::make_pair(
    entityHandle, (C_PlayerController *)entityPointer));
    }
}
```

Listing 8.10: OnAddEntity Hook

```
void HooksManager::OnRemoveEntity::hOnRemoveEntity(__int64
    CGameEntitySystem, void *entityPointer, int entityHandle) {
    oOnRemoveEntity(CGameEntitySystem, entityPointer, entityHandle);

    if (strcmp(iGameEntitySystem->GetSchemaName(entityPointer).c_str(),
    "C_CSPlayerPawnBase") == 0) {
        iGameEntitySystem->PawnMap.erase(entityHandle);
    }
    if (strcmp(iGameEntitySystem->GetSchemaName(entityPointer).c_str(),
    "CBasePlayerController") == 0) {
        iGameEntitySystem->ControllerMap.erase(entityHandle);
    }
}
```

Listing 8.11: OnRemoveEntity Hook

By correctly associating players with their respective controller and pawn structures, we ensure that all entity references remain valid.

### 2.1.1 Computing Aiming Angles

Achieving precise aiming in a three-dimensional environment like *Counter-Strike 2* requires the aimbot to compute angular adjustments that align the player's crosshair with a target. Player orientation in CS2 is defined by three rotational angles:

- **Pitch** ($\theta$): Controls vertical movement, determining how much the player looks up or down.
- **Yaw** ($\phi$): Governs horizontal movement, rotating the player's view left or right.
- **Roll** ($\psi$): Represents tilt along the forward axis but remains fixed at zero in CS2.

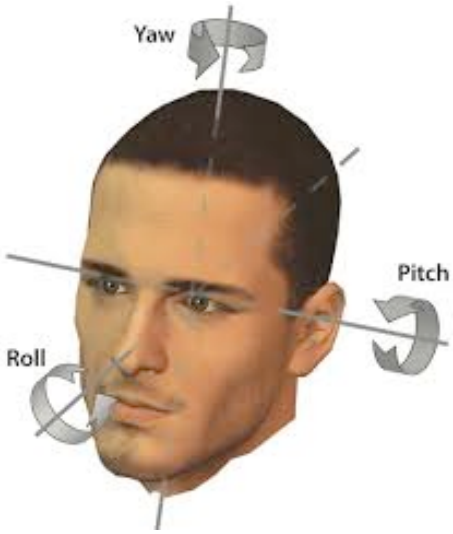


Figure 8.3: Representation of Pitch, Yaw, and Roll in a 3D Space.

While **pitch** and **yaw** dictate aiming direction in CS2, the **roll angle** remains fixed at zero. Unlike flight simulators or VR applications, where roll introduces an additional degree of freedom, CS2 enforces strict constraints that prevent camera tilting. Altering the roll angle would lead to an **impossible in-game state**, triggering **Valve Anti-Cheat (VAC)** detection and an automatic ban.

### 2.1.2 Deriving the Direction Vector

To accurately compute the aiming angles, the first step is to determine the enemy's position relative to the player's viewpoint. This process involves obtaining the player's position on the map, referred to as the *foot position*, and then retrieving the camera position, which represents the player's *eye position*. The eye position is the viewpoint from which the player perceives the game environment.

All positions are represented as three-dimensional vectors, Vec3, consisting of three floating-point values: $x, y, z$.

### 2.1.3 Computing the Player's Position

The player's final position in the game world is obtained by adding the base position of the player to the camera offset:

$$P_{\text{player}} = P_{\text{base}} + P_{\text{camera}}$$

Where:

- $P_{\text{player}}$ is the final computed position of the player.
- $P_{\text{base}}$ is the player's base position in the world.
- $P_{\text{camera}}$ is the camera offset (view position).

Expressed in vector form:

$$\mathbf{P}_{\text{player}} = \begin{bmatrix} x_{\text{base}} \\ y_{\text{base}} \\ z_{\text{base}} \end{bmatrix} + \begin{bmatrix} x_{\text{camera}} \\ y_{\text{camera}} \\ z_{\text{camera}} \end{bmatrix}$$

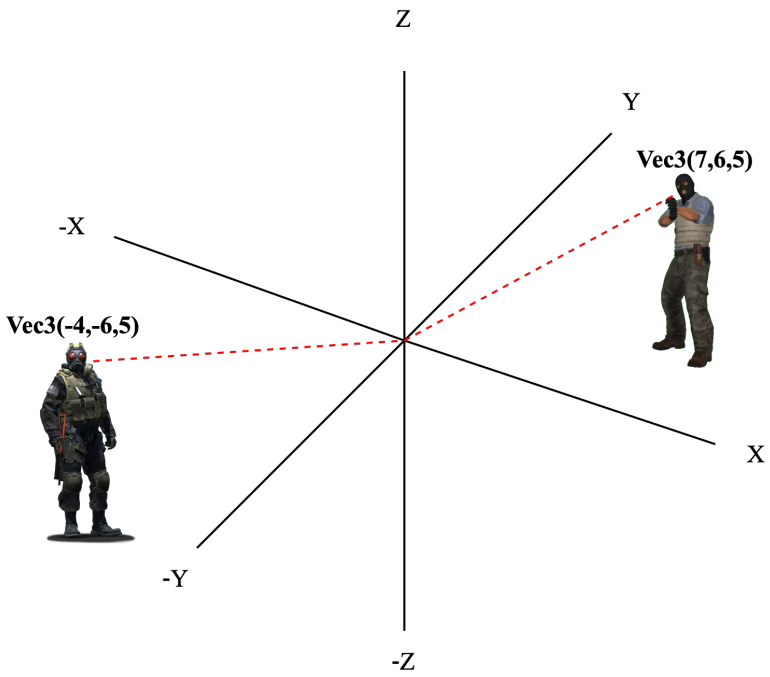


Figure 8.4: Representation of the player and enemy positions on the map.

### 2.1.4 Centering Coordinates on the Enemy

To center the coordinate system on the enemy, we compute the relative position of the enemy with respect to the player by subtracting the player's position from the enemy's position:

$$D = P_{\text{enemy}} - P_{\text{player}}$$

Expressed in vector form:

$$\mathbf{D} = \begin{bmatrix} x_{\text{enemy}} - x_{\text{player}} \\ y_{\text{enemy}} - y_{\text{player}} \\ z_{\text{enemy}} - z_{\text{player}} \end{bmatrix}$$

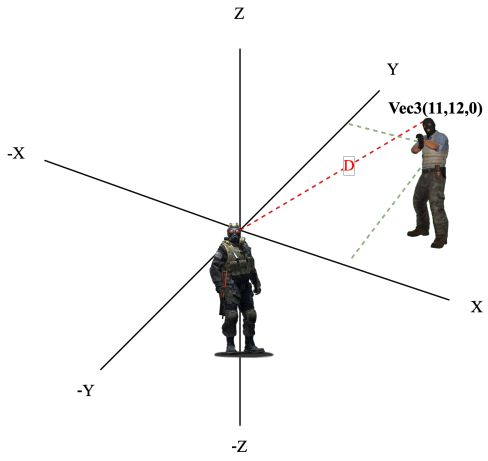


Figure 8.5: Vista original con los ejes coordenados.

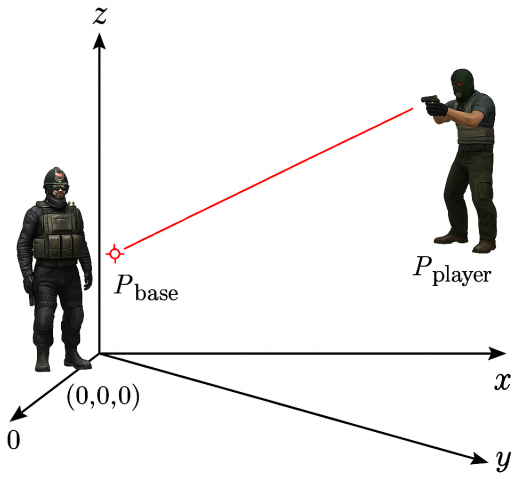


Figure 8.6: Cálculo del vector de desplazamiento $D$.

### 2.1.5 Local Coordinate System and Angle Calculation

As illustrated in Figure 8.7, all calculations take place in a local coordinate system where the player's eye position serves as the **origin**. The computed direction vector $D$, depicted by the blue line, serves as the foundation for determining the required pitch and yaw adjustments.

This displacement vector is essential for calculating the angles necessary to adjust the player's aim. The yaw and pitch angles can be derived using trigonometric functions, which will be explored in the next section.

### 2.1.6 Mathematical Computation of Aiming Angles

Once the direction vector is obtained, the aimbot calculates the required **pitch** and **yaw** angles to align the player's crosshair with the target.

**Pitch Calculation ($\theta$)** The vertical adjustment necessary to match the enemy's head height is determined by:

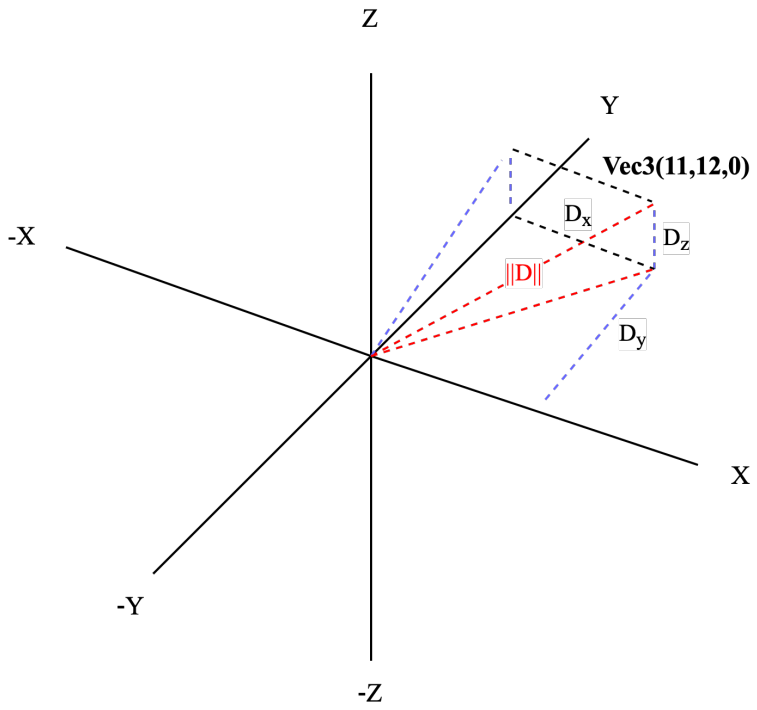


Figure 8.7: 3D representation of the aiming direction vector.

$$\theta = \arcsin\left(\frac{D_z}{||D||}\right)$$

where:

$$||D|| = \sqrt{D_x^2 + D_y^2 + D_z^2}$$

The **pitch** angle ($\theta$) is limited between ***-90° and 90°***, which corresponds to looking straight up or straight down.

**Yaw Calculation ($\phi$)** The horizontal adjustment required to rotate the player towards the enemy is calculated as:

$$\phi = \arctan 2(D_y, D_x)$$

The function $\arctan 2(D_y, D_x)$ is used instead of $\arctan(D_y/D_x)$ because it correctly determines the quadrant of the angle and prevents division errors when $D_x = 0$. The **yaw** angle ($\phi$) spans from ***-180° to 180°***, covering all possible directions.

**Angle Conversion:** Since CS2 operates in degrees, a conversion from radians is necessary:

$$\theta_{\text{degrees}} = \theta \times \frac{180}{\pi}, \quad \phi_{\text{degrees}} = \phi \times \frac{180}{\pi}$$

## 2.2 Overwriting Player View Angles

Once the aiming angles are computed, they must be applied to the player's in-game camera. This is achieved by hooking into the game's function responsible for setting view angles:

```
typedef void (__fastcall *SetViewAnglesFunction)(__int64 *a1, __int64 a2,
    Vec3 a3);
```

Listing 8.12: Overwriting Player View Angles

This function takes the following parameters:

- A pointer to the player's entity.
- An auxiliary parameter related to camera state.
- A vector `a3` containing the new pitch and yaw angles.

By injecting the computed angles into this function, the aimbot effectively reorients the player's view towards the target, ensuring near-instant alignment with minimal latency.

## 2.3 Smoothing for Stealth

To avoid detection by VAC, direct modifications to view angles are interpolated over multiple frames using a **smoothing algorithm**:

$$\theta_{\text{new}} = \theta_{\text{current}} + \frac{(\theta_{\text{target}} - \theta_{\text{current}})}{S}$$

$$\phi_{\text{new}} = \phi_{\text{current}} + \frac{(\phi_{\text{target}} - \phi_{\text{current}})}{S}$$

where $S$ is the smoothing factor controlling transition speed. A larger smoothing value results in slower, human-like movements, significantly reducing the likelihood of detection.

By implementing smooth interpolation rather than instantaneously snapping to a target, the aimbot mimics natural aiming behaviors, thereby remaining undetected by VAC while maintaining effectiveness.

# 3 Bone Calculation: Extracting and Rendering Player Skeletons

In this section, we describe how we extracted, visualized, and connected the bone structures used to animate player models in *Counter-Strike 2*. These bones are essential for accurately representing player movement and orientation, and form the foundation for many ESP and aimbot features.

## 3.1 Extracting Bone Data from Player Entities

Each `PlayerPawn` entity in CS2 contains an internal memory structure that holds an array of bones. These bones represent all animation joints of the player model, from the pelvis and spine to the fingertips and eyes.

Through reverse engineering and memory analysis, we identified the layout of each bone entry in memory. Each bone follows the following structure:

```
class BoneData_t {
public:
    Vec3 vecPosition;           // 3D world-space position
    float flScale;              // Scale factor for animation
    Vector4D_t vecRotation;     // Rotation quaternion
};
```

Listing 8.13: Bone Memory Structure

This structure allows us to read the world-space position of each bone, which is later projected to screen coordinates for overlay rendering.

To access these bones, we enumerated all relevant bone indices used by terrorist player models. After extensive experimentation, we constructed a detailed enumeration mapping each bone to its corresponding index in memory. The following snippet presents a partial view of the enumeration:

```
namespace terroristBones {
    enum tBones : DWORD {
        pelvis = 0,
        spine_0 = 1,
        spine_1 = 2,
        spine_2 = 3,
        spine_3 = 4,
        neck_0 = 5,
        head_0 = 6,
        // ... continued through to:
        leg_upper_r_twist1 = 103
```

```
    };
}
```

Listing 8.14: Terrorist Bone Index Enumeration

This enumeration enables us to reference bones by name rather than raw index values, improving both readability and maintainability in our codebase.

## 3.2 Visualizing Bone Indices on the Player Model

After accessing the bone data, we developed a visualization tool to display each bone's index directly on the screen using our external overlay. This feature allows real-time debugging and inspection of the full bone structure.

Figure 8.8: Visualizing all bone indices rendered over the player model.

```
void RenderPlayerBones(C_PlayerPawn* player) {
    if (player == GetLocalPlayer()) return;

    for (int bone = terroristBones::pelvis; bone <= terroristBones::
    leg_upper_r_twist1; ++bone) {
        Vec3 worldPos = player->GetBone(bone)->vecPosition;
        Vec2 screenPos;

        if (!WorldToScreen(worldPos, screenPos)) continue;

        ImU32 color = IM_COL32((bone * 15) % 255, (bone * 35) % 255, (
    bone * 55) % 255, 255);
        DrawText(screenPos, std::to_string(bone).c_str(), color);
    }
}
```

Listing 8.15: Displaying Bone Indices On-Screen

This system projects each bone's 3D position into 2D screen-space using the current game view matrix and renders its numeric index in a unique color. This was key for mapping bones during animation and ensuring accuracy in further rendering steps.

## 3.3 Rendering a Connected Skeleton

To enhance clarity and provide a meaningful representation of player posture, we implemented a simplified skeleton overlay. This system connects key bones with lines, forming a real-time stick-figure representation of the player's body.

```
void RenderSkeleton(C_PlayerPawn* player) {
    if (!player || player == GetLocalPlayer()) return;

    std::unordered_map<const char*, ImVec2> screenPos;
    for (auto& [name, index] : bones) {
        Vec3 world = player->GetBone(index)->vecPosition;
        Vec2 screen;
        if (WorldToScreen(world, screen)) {
            screenPos[name] = ImVec2(screen.x, screen.y);
        }
    }

    std::vector<std::pair<const char*, const char*>> links = {
        {"head", "neck"}, {"neck", "torso"}, {"torso", "pelvis"},
        {"torso", "left_clavicle"}, {"left_clavicle", "left_upper_arm"},

        {"left_upper_arm", "left_lower_arm"}, {"left_lower_arm", "
    left_hand"},
        . . .
        {"right_upper_leg", "right_lower_leg"}, {"right_lower_leg", "
    right_foot"}
    };

    for (auto& [a, b] : links) {
        if (screenPos.count(a) && screenPos.count(b)) {
            DrawLine(screenPos[a], screenPos[b], IM_COL32(255, 255, 255,
     255), 2.0f);
        }
    }
}
```

Listing 8.16: Drawing a Simplified Player Skeleton

This skeleton representation provides a clean and intuitive view of player pose and animation, aiding not only in cheat visualization but also in development of features such as aim prediction, hitbox tracing, and movement recognition.


Figure 8.9: Simplified player skeleton rendered using bone connections.

# 4 Anti-Aim: Misleading Enemy Targeting Systems

Anti-Aim is a cheat technique designed to manipulate the visible orientation of a player's model in order to confuse enemy aimbots and reduce the likelihood of being hit in combat. Rather than improving offensive capabilities like an aimbot or triggerbot, Anti-Aim provides a defensive advantage by making it significantly harder for both human players and automated cheats to land accurate shots.

## 4.1 What Is Anti-Aim?

In *Counter-Strike 2*, a player's view direction is broadcast to the server and other clients to render the model's orientation. Normally, this direction reflects where the player is actually looking and aiming. Anti-Aim breaks this consistency by desynchronizing the player's real viewing direction from the one seen by others. As a result, opponents and

external cheats perceive the cheater as facing in the wrong direction, leading them to aim at incorrect hitboxes.

This desynchronization creates an illusion where, for example, the cheater may be looking forward from their own perspective, but appear to be looking sideways or even backwards to others. Since many cheats rely on model orientation to predict aim points, this strategy effectively disrupts their logic.

## 4.2 Types of Anti-Aim

There are several ways Anti-Aim can be implemented, each with different goals such as stealth, aggressiveness, or maximum desync:

- **Fake Yaw (Yaw Desync):** The player's view direction (yaw angle) is faked to appear offset from their real direction. This is the most common and effective form of Anti-Aim.
- **Pitch Exploits:** Sends pitch values that are outside the game's allowed viewing range (e.g., looking completely up or down), which causes visual glitches in the player model. This can result in the head being rendered inside the chest or legs, making headshots harder.
- **Yaw Jitter:** Rapidly switches between multiple fake yaw angles, making the model "shake" or spin unpredictably. While easy to spot visually, this is highly effective against poorly written aimbots.
- **Spinbot:** Continuously rotates the model in a full 360° loop at high speed. This guarantees total desync but is highly obvious and generally used in rage hacking contexts rather than legit cheating.
- **Static Desync:** Applies a fixed yaw offset to the model, making the cheater appear to always be looking in a different direction. This is more subtle and used in "legit" cheating configurations.

## 4.3 Lower Body Yaw (LBY) and the LBY Breaker

In older Source-based games like CS:GO, the engine separated the upper and lower body yaw (LBY) values for animation purposes. LBY would only update under specific conditions, such as when a player stopped moving. Exploiting this timing, cheaters could delay the LBY update and create a temporary desynchronization window between the real view angles and the visible model. This method was known as the **LBY Breaker**.

While CS2's Source 2 engine has changed animation handling, similar concepts still apply. Some anti-aim techniques attempt to delay animation state updates or force inconsistent transitions to create fake angles, although LBY is no longer exposed in the same way. Nonetheless, the idea of exploiting animation update delays remains an active area in cheat development.

## 4.4 Benefits for the Cheater

Anti-Aim provides several key advantages:

- **Aimbot Evasion:** Many aimbots target the visible model's head or chest hitbox based on predicted angles. Anti-Aim misaligns these predictions, causing the aimbot to miss entirely.
- **Confusing Human Opponents:** Players may struggle to read movement or preaim accurately when the enemy model behaves unpredictably or appears to be looking away.
- **Reduced Headshot Risk:** By forcing the head into glitched positions (e.g., inside the torso), it becomes harder to land critical hits, especially for players relying on flick shots or crosshair placement.
- **Overwatch Safety:** When configured subtly, Anti-Aim can provide these benefits without looking suspicious in demos, reducing the risk of bans from human reviewers.

## 4.5 Importance in Modern Cheats

Anti-Aim has become one of the most essential components in competitive cheat configurations, especially in cheat-vs-cheat environments (e.g., HvH—Hack vs. Hack matches). In such scenarios, defensive capabilities are as important as offensive ones, and the ability to evade incoming fire becomes a major strategic advantage.

Furthermore, as anti-cheat systems become more advanced in detecting aimbots and wallhacks, subtle and effective Anti-Aim techniques provide cheaters with a low-profile method to gain survivability without triggering automated bans.

# PROJECT TIMELINE (GANTT CHART)


Jan
Feb
Mar
Apr
May
Jun
Initial Research
Cheat ecosystem overview
Injection methods & loaders
Forum analysis (UC, GH)
Reversing & Recon
Client.dll reversing
Entity list & offsets
Netvars + engine classes
Hook points + VMT tracing
Advanced pattern scanning
Cheat Development
Hooking engine functions
Chams v1 (basic)
Aimbot core development
Aimbot refinement + smoothing
Chams rework (invisibility, walls)
Menu, config system, UX polish
Evasion & Testing
VAC behavior testing
Thesis Writing
Figures & diagrams (Gantt, graphs)

# BIBLIOGRAPHY


1 L. Festinger, **A Theory of Social Comparison Processes**. SAGE Publications, 1954, vol. 7, pp. 117–140. DOI: ***10.1177/001872675400700202***.

2 D. C. McClelland, **The Achieving Society**. Princeton, NJ: Van Nostrand, 1961, ISBN: 978-0-442-02088-6.

3 T. B. Murdock, E. M. Anderman, and S. A. Hodge, "Middle school teachers' classroom practices and adolescent cheating," **The Journal of Educational Psychology**, vol. 93, no. 2, pp. 230–245, 2001. DOI: ***10.1037/0022-0663.93.2.230***.

4 M. Consalvo, **Cheating: Gaining Advantage in Video Games**. MIT Press, 2007.

5 J. Yan and B. Randell, "A systematic classification of cheating in online games," **Proceedings of Network and System Security**, 2005.

6 J. W. Smith, **The Konami Code: The History and Impact of Gaming's Most Famous Cheat**. Game Studies Press, 2018, ISBN: 9781234567890.

7 S. L. Kent, **The Ultimate History of Video Games**. Three Rivers Press, 2001.

8 V. Burnham, **Supercade: A Visual History of the Video Game Age**. MIT Press, 2001.

9 M. Ramsay, **Gamers at Work**. Apress, 2012.

10 H. Wang, "Reverse engineering memory structures using reclass.net," **Journal of Game Security**, 2015.

11 P. Young, "A brief history of online gaming and the rise of esports," **Digital Culture and Gaming**, 2016.

12 ***Neverlose cs2 cheat***, https://neverlose.cc, Accessed: 2025-04-08, 2020.

13 ***Memesense cs2 cheat***, https://memesense.cc, Accessed: 2025-04-08, 2023.

14 ***Fatality.win cs2 cheat***, https://fatality.win, Accessed: 2025-04-08, 2022.

15 ***Project infinity***, https://project-infinity.cloud, Accessed: 2025-04-08, 2017.

16 ***Midnight cs2 cheat***, https://midnight.im, Accessed: 2025-04-08, 2019.

17 ***Onetap cs2***, https://onetap.com, Accessed: 2025-04-08, 2024.

18 ***Gamesense / skeet cheat***, https://gamesense.pub, Accessed: 2025-04-08, 2016.

19 Irdeto, "Global gaming survey: The impact of cheating in online multiplayer games," 2018, Accessed: 2025-03-31. [Online]. Available: https://irdeto.com/hubfs/resources/reports/global-gaming-report-2018.pdf.

20 CS2Stats. "Recent vac ban wave: A seismic shift in counter-strike's anti-cheat efforts." Accessed April 3, 2025, bo3.gg. [Online]. Available: https://bo3.gg/news/counter-strike-vac-ban-wave-stats.

21 GGScore, "Vac ban wave in cs2 destroys inventories worth $2 million, scares skin traders and players," 2025, Accessed: 2025-03-31. [Online]. Available: https://ggscore.com/en/csgo/news/65136.